Dynamic Knowledge-Driven Enterprise Cybersecurity Topology Platform

System Design — v3

Dan Lenyel

Status: For design review

Contents

Dynamic Knowledge-Driven Enterprise Cybersecurity Topology Platform

System Design — v3 (adds Zero-Day Triage and Supply Chain Risk use cases) Author: Dan Lenyel Status: For design review

> **Changes vs v2**: Zero-Day Triage and Supply Chain Risk are added as first-class use cases. New ontology nodes (Indicator, Advisory, ThreatActor, TTP, Package, SBOM, BuildArtifact, ThirdParty), new data sources (threat intel feeds, SCA tools, package registries, source repos, CI/CD, container registries, code-signing, TPRM), new agents (Zero-Day Triage, Supply Chain Risk), new failure modes (TI poisoning, IOC false-positive storm, SBOM staleness, transitive-dep explosion), new runbooks (zero-day declared, supplier compromise), and a re-phased roadmap. The fuzzy-matching path for IOCs is treated as distinct from classic ER. Transitive dependency closure introduces a new query type that breaks the 6-hop traversal cap with explicit cost guards.

---

1. TL;DR

Enterprise cybersecurity teams operate without a unified, relationship-aware view of their environment that is fresh enough to act on. CMDB, scanners, PKI, identity, cloud, ticketing, threat intel, SBOM, and build provenance systems each answer narrow questions; none answers "which internet-facing production apps in Consumer Banking have critical vulnerabilities chained to restricted data, who owns them, and which of those depend transitively on a package just disclosed as compromised?" That question is answered today by an analyst with twelve tabs open and a spreadsheet — slowly, expensively, and with no audit trail.

This document proposes a multi-tenant **Cyber Knowledge Topology Platform** on OpenShift private cloud. Polyglot persistence with Neo4j (topology + transitive dependency closure), BigQuery/Postgres (structured analytics & metadata), a vector store (semantic context), OpenSearch (text/log search + IOC matching), and an object store (evidence/replay). A **governed query orchestration layer** routes natural language questions to graph, SQL, vector, indicator-match, or hybrid execution paths. A **supervised agent runtime** provides specialized agents — Topology, Vulnerability Chain, PKI, Ownership, **Zero-Day Triage**, **Supply Chain Risk** — constrained to typed APIs and tenant-scoped tools.

Top four architectural bets:

1. **Ontology and entity resolution are the product.** If ER quality drops, every downstream answer is wrong. We invest disproportionately in ER, ontology governance, and bitemporal modeling before adding use cases.

2. **Agents call governed APIs only; never raw stores.** Capability containment is the load-bearing security control. Specialist agents have narrow tool surfaces; a supervisor orchestrates and is the only context-aware actor.
3. **Composite tenancy with mandatory query rewriting.** Single shared graph for the common case with mandatory `tenant_id` predicate injection at the planner, plus separate Neo4j databases for high-sensitivity domains. Cross-tenant queries — including burst patterns triggered by enterprise-wide zero-days — are first-class and gated.
4. **Two fundamentally different query shapes are first-class.** Bounded multi-hop traversal (vuln chains, PKI impact) and unbounded transitive closure (dependency reach). The platform treats them as distinct execution modes with different cost models, cap strategies, and pre-computation paths.

Top risks and mitigations:

Risk	Mitigation
Entity resolution quality	Tiered confidence (auto-merge / review queue / no-link), bitemporal edges, survivorship registry, weekly ER eval against gold set
Multi-tenant authorization leakage	Defense in depth: gateway → policy decision → query rewriter → store enforcement → result filter; adversarial tenant-boundary test suite is a release gate
Model risk (NL2SQL, embeddings, scorers, IOC matchers, supply chain risk)	SR 11-7 tier mapping per model, validation per tier, drift detection within 24 hours, kill switch per model
Ontology drift across teams	Single ontology owner, SemVer with deprecation windows, mandatory CAB review, ontology version stamped on every node/edge/result
Threat intel false positives at scale	Confidence + decay model per indicator, source-trust scoring, suppression by analyst feedback, IOC storm circuit breaker
Supply chain transitive blast radius	Pre-computed dependency closure for hot packages, depth-capped graph traversal with explicit cost guards, asynchronous deep-closure queries with reserved capacity

Ask: Approve Phase 0 (8 weeks) foundation + Phase 1 (14 weeks) MVP across CMDB + Vuln Scanner + PKI + Ownership + **SBOM/SCA** with three governed use cases (Vulnerability Chain, PKI Risk, **Supply Chain Risk**). Zero-Day Triage lands in Phase 2 after TI infrastructure is in place.

---

2. Problem statement & goals

Problem

Cybersecurity-relevant data is fragmented across at least fifteen enterprise systems. Each system answers isolated questions accurately; none answers questions that traverse the relationships *between* them. Analysts compensate manually, which is slow, error-prone, and does not scale to enterprise-grade incident response, zero-day triage, or supply chain risk management.

Current state	Consequence
Fragmented inventory across CMDB, scanners, cloud, K8s, endpoint	No authoritative answer to “what exists and who owns it”
Flat vulnerability lists ranked by CVSS only	CVSS does not encode exposure, data sensitivity, or business impact
Weak ownership mapping	MTTR inflated; tickets bounce; SLAs missed
Limited certificate impact visibility	PKI expirations cause outages
Zero-day response is a manual war room	When a vendor advisory drops, hours are spent on “do we have this?” — answers are partial and unverified
Supply chain risk is opaque below direct dependencies	When `xz-utils` or `event-stream` is compromised, finding transitive exposure takes days; we can’t reliably answer “which of our images contain this package”
Manual correlation across tools	Analyst hours spent on join logic that should be data infrastructure
Point-in-time reports, not real-time topology	Reports are stale by the time leadership reads them
Inconsistent access controls across tools	Auditor can’t defensibly answer “who can see what”

Goals (measurable)

Goal	Outcome
Unified cyber topology	≥90% of in-scope assets linked to owner, app, environment, tenant, source
Vulnerability chaining	Identify exploitable paths across exposure → vulnerability → service → data class
PKI intelligence	Expiring/weak certs mapped to apps, APIs, endpoints, owners with downstream impact
Zero-day triage	From advisory ingestion to ranked exposure list in < 15 min p95; precision@20 ≥ 0.85 against SME-labeled benchmark
Supply chain reach	Full transitive dependency closure for a named package version returned in < 30 s p95 at Phase 1 scale; SBOM coverage ≥ 90% of production images
Multi-tenant isolation	100% pass on adversarial tenant-boundary test suite; zero leaks in audit
Natural language querying	Governed graph, SQL, vector, indicator-match, hybrid execution with full audit
Tiered freshness	T1 critical changes < 1 min p95; T2 full topology updates < 15 min p95; T3 bulk recompute < 1 hr p95
Visual exploration	Topology graph, dependency view, impact subgraphs, risk dashboards, evidence panel with lineage

Non-goals

- **Do not replace systems of record.** CMDB, vulnerability scanner, PKI, SIEM, ticketing, SCA tools (Black Duck/Snyk), and threat intel platforms remain authoritative for their respective data. This platform reads from them, links them, and governs queries over them — it does not try to become any of them.
- **Do not produce threat intelligence.** We consume from approved feeds and registries; we do not generate or distribute IOCs externally.
- **Do not build behavioral detection.** We consume detections from SIEM/EDR; we do not author detection rules or run a detection engine.
- **Do not replace SBOM scanners.** We consume their output (CycloneDX/SPDX); we do not perform composition analysis ourselves.
- **Do not perform automated remediation.** No automated patching, package upgrades, ticket closure, certificate rotation, or remediation execution in MVP or Phase 1. The platform recommends; humans and existing remediation systems act.
- **Do not expose unrestricted Cypher or SQL.** Users, analysts, and agents access data through governed templates, NL2SQL with AST validation, or use-case APIs only.
- **Do not cover hardware or firmware supply chain in Phase 1–2.** Firmware is a Phase 3+ candidate; hardware supply chain is out of scope.
- **Do not aim for complete source coverage in Phase 1.** Phase 1 covers a deliberate subset (see §22); broader coverage is phased in.
- **Do not build an enterprise data lake.** This platform is purpose-built for cyber topology; bulk analytics outside that scope belong to the existing data platform.
- **Do not grant agents production write access to any system** in MVP or Phase 1. Read-only against all stores; writes (e.g., ticket creation) go through approved use-case APIs with HITL gates.

Personas and jobs-to-be-done

Persona	JTBD	Primary surface
Security Analyst	Find high-risk vulnerabilities prioritized by topology and business impact	Topology UI + NL query
Application Owner	Understand assets, certs, vulns, packages belonging to my app	Owner dashboard
PKI Team	Identify expiring/weak certs and impacted applications	PKI dashboard
Vulnerability Mgmt	Prioritize remediation by exploitability and blast radius	Vuln Chain use case
Threat Intel Analyst	Triage incoming advisories; assess organizational exposure	Zero-Day Triage console
Software Supply Chain Owner	Track third-party software risk, SBOM health, build provenance	Supply Chain dashboard
Incident Response Lead	During an incident, rapidly answer “where else” and “what’s connected”	Cross-cutting NL query + impact subgraph
Cyber Leadership	View posture by BU, owner, app, tenant; cross-tenant rollups	Leadership dashboard + cross-tenant APIs

Persona	JTBD	Primary surface
Auditor / Risk	Review evidence, lineage, access logs, control coverage	Audit & evidence surfaces

---

3. Context & constraints

Constraint area	Design implication
Model risk (SR 11-7)	Every model has tier, owner, validation, monitoring; changes are gated
Data residency	On approved private cloud / on-prem; threat intel ingestion only from approved feeds
Tenant boundary enforcement	Every query path enforces tenant scope; cross-tenant access is a separate, audited gate including burst patterns
Auditability	Immutable audit of question → resolved scope → generated query → data accessed → result emitted, hash-chained
Explainability	Risk scores, graph paths, IOC matches, and answers ship with evidence + lineage
Approved tooling	OpenShift; approved DBs; Vault; approved TI feeds; approved SCA tools
Performance	Common queries bounded by latency and cost budgets; transitive closure queries get dedicated async capacity
Change management	CR process, canary deployments, rollback on policy/eval regression

Source systems in scope (phased):

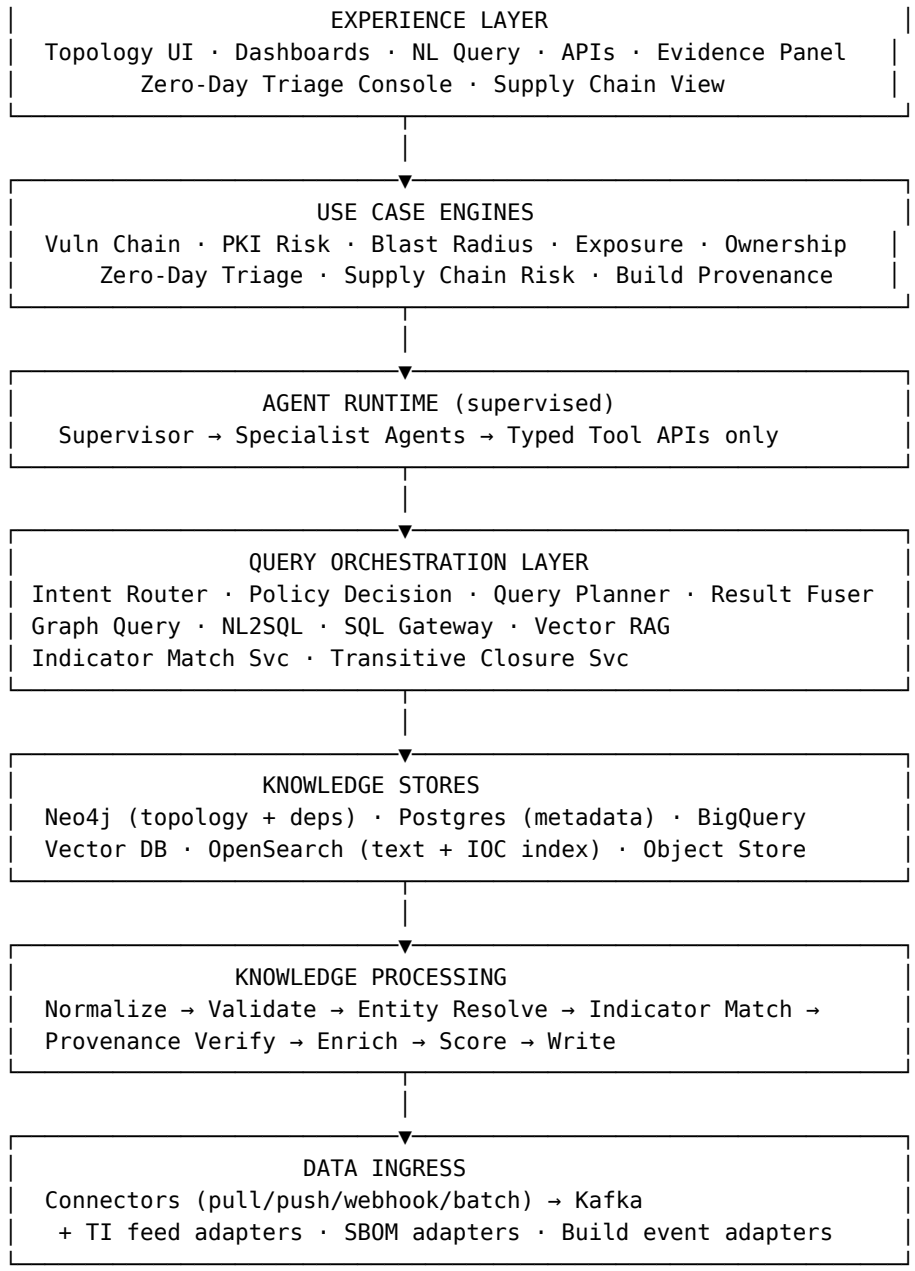
Existing categories: ServiceNow CMDB, Qualys/Tenable/Rapid7, CrowdStrike/Tanium/SCCM, Venafi/DigiCert/internal CA, AD/Azure AD/Okta, DNS/IPAM, firewalls, load balancers, AWS/Azure/GCP inventories, Kubernetes/OpenShift, Jira/ServiceNow Tickets, data catalogs, Splunk/SIEM, OpenSearch logs.

New for zero-day: Approved threat intel feeds (commercial + government — CISA KEV, vendor advisories), MISP/OpenCTI internal aggregator, CVE feeds (NVD, vendor PSIRTs), exploit databases, MITRE ATT&CK TTP mappings, SIEM/EDR for IOC matching.

New for supply chain: SBOM scanners (Black Duck, Snyk, Trivy, Syft outputs), private package registries (Artifactory, Nexus), source control (GitHub Enterprise, GitLab), CI/CD systems (Jenkins, GitHub Actions, internal build platform), container registries (Harbor, internal), code-signing infrastructure, build provenance (in-toto attestations, SLSA), third-party risk management (TPRM platform), vendor inventory.

---

4. High-level architecture



Layer	Owns	Explicitly does not own
Experience	UI, dashboards, NL surface, APIs to consumers	Authorization decisions
Use Case Engines	Domain logic incl. Zero-Day and Supply Chain	Raw data, generic query routing

Layer	Owns	Explicitly does not own
Agent Runtime	Supervisor + specialist orchestration, typed delegation	Direct store access, raw SQL/Cypher
Query Orchestration	Intent routing, planning, fusion, store-specific query services, indicator match, transitive closure	Domain semantics
Knowledge Stores	Persistent state across polyglot stores	Authorization (defense in depth only)
Knowledge Processing	Normalization, ER, indicator match, provenance verification, enrichment, scoring, writing	User-facing APIs
Data Ingress	Source connection, raw event publication; TI/SBOM/build adapters	Canonical modeling, scoring
Cross-cutting	Identity, policy, audit, registries, evals, kill switches, TI source trust	Business logic

---

5. Logical architecture / component model

The Phase column shows when each component first becomes operational. Components marked "Phase 0/1 scaffolded → Phase N live" have their interfaces and persistence laid down in Phase 0 or 1 (so other components can be built against them) but only become functionally live in the named later phase.

Component	Responsibility	Interface	Runtime	Phase
Source Connector Service	Pull/push from sources via approved protocols	Connector SDK	Horizontally scalable	**MVP** (Phase 1)
SBOM Adapter	Ingest CycloneDX/SPDX from SCA tools; resolve to internal artifacts	SBOM ingestion API	Horizontally scalable	**MVP** (Phase 1)
Build Event Adapter	Ingest build/deploy events with image/artifact metadata; SLSA/in-toto attestation processing	Build ingestion API	Horizontally scalable	**MVP** basic (Phase 1) → Phase 3 full attestation

Component	Responsibility	Interface	Runtime	Phase
TI Feed Adapter	Normalize threat intel from approved feeds; apply source trust score	TI ingestion API	Horizontally scalable	**Phase 2**
Event Bus	Stream cyber events with replay	Kafka topics	HA cluster	**MVP** (Phase 0)
Normalization Service	Source payload → canonical schema per ontology version	Internal event contract	Stateless	**MVP** (Phase 0)
Validation Service	Schema validation, sanity checks, quarantine on failure	Internal contract	Stateless	**MVP** (Phase 0)
Entity Resolution Service	Block, match, score, link, manage survivorship	ER API	Hybrid stateful	**MVP** (Phase 0 framework, Phase 1 live with sources)
Indicator Matching Service	Fuzzy match indicators (CPEs, hashes, IPs, domains, behaviors) against environment	Match API	Stateless workers + OpenSearch index	**Phase 0 scaffolded** (interfaces, indexes) → **Phase 2 live**
Provenance Verifier	Verify in-toto/SLSA attestations, code signatures, build pipeline integrity	Verify API	Stateless	**MVP** signature verification only (Phase 1, advisory mode) → **Phase 3** enforcement at deploy gate
Enrichment Service	Add derived attributes (exploitability, exposure class, threat actor attribution)	Internal contract	Stateless	**MVP** baseline (Phase 1) → Phase 2 adds threat-actor enrichment
Risk Scoring Service	Deterministic risk scoring with explainable features	Scoring API	Stateless	**MVP** (Phase 1)

Component	Responsibility	Interface	Runtime	Phase
Supply Chain Risk Scorer	Compute supply chain risk: dependency presence × runtime exposure × data class × business criticality	Scoring API	Stateless	**MVP** (Phase 1)
Zero-Day Triage Scorer	Compute probability_affected from partial advisory signal + environment match	Scoring API	Stateless	**Phase 2**
Graph Writer	Idempotent upserts to Neo4j with bitemporal edges	Internal write contract	Stateless workers	**MVP** (Phase 0)
Structured Writer	Writes to BigQuery / Postgres analytics tables	Internal write contract	Stateless	**MVP** (Phase 0)
Embedding Pipeline	Create embeddings for docs/entities, version-aware	Embedding job API	Batch + streaming	**MVP** (Phase 1)
Transitive Closure Service	Pre-compute and serve dependency closure; bounded synchronous + async deep	Closure API	Stateless workers + materialized views	**MVP** bounded sync + top-1% pre-mat (Phase 1) → **Phase 3** full materialization
Ontology Registry	Versioned ontology catalog, change control	Registry API	HA	**MVP** (Phase 0)
Schema Registry	Versioned source schemas, mapping rules	Registry API	HA	**MVP** (Phase 0)
Model Registry	LLMs, embedding models, scorers; version, owner, tier, status	Registry API	HA	**MVP** (Phase 0)
TI Source Trust Registry	Per-feed trust score, freshness, false-positive rate, last-verified	Registry API	HA	**Phase 0 scaffolded** (registry plumbing, no live feeds) → **Phase 2 live**
Policy Service	Resolve user scope, evaluate ABAC/RBAC/ReBAC	PDP API	HA	**MVP** (Phase 0)

Component	Responsibility	Interface	Runtime	Phase
Intent Router	Classify query intent; route to executor(s)	`/query/intent`	Stateless	**MVP** (Phase 1)
Query Planner	Build execution plan; inject mandatory predicates	Internal	Stateless	**MVP** (Phase 1)
Graph Query Service	Orchestrate graph queries through six deterministic internal stages: intent classify → path select → (template or NL2Cypher) → Cypher Gateway → subgraph serialize → result fusion adapter. No internal LLM agents. See §16.2	`/query/graph`	Stateless	**MVP** (Phase 1)
NL2Cypher Service	Constrained Cypher generation with schema-aware prompting for free-form topology questions. Templates preferred whenever possible. Mirrors NL2SQL governance posture	Internal (invoked by Graph Query Service stage 3)	Stateless	**MVP** (Phase 1, baseline) → **Phase 2** scale-up

Component	Responsibility	Interface	Runtime	Phase
Cypher Gateway	AST validate, mandatory tenant predicate inject on every matched node, reject banned constructs (unbounded paths, Cartesian on large labels, arbitrary `apoc.cypher.run`), inject depth + LIMIT caps, cost estimate (PROFILE), gate decision, execute read-only with timeout, post-process classification masking + final tenant filter. The load-bearing graph control. Symmetric to SQL Gateway	Internal (invoked by Graph Query Service stage 5)	Stateless	**MVP** (Phase 1)
NL2SQL Service	Constrained SQL generation with schema-aware prompting	`/query/sql/generate`	Stateless	**MVP** (Phase 1)
SQL Gateway	AST validate, predicate inject, cost estimate, execute read-only	`/query/sql/execute`	Stateless	**MVP** (Phase 1)
Vector RAG Service	Tenant-scoped semantic retrieval and reranking	`/query/vector`	Stateless	**MVP** (Phase 1)
Result Fusion Service	Merge graph + SQL + vector + indicator-match outputs with provenance	Internal	Stateless	**MVP** baseline (Phase 1, graph+SQL+vector) → **Phase 2** adds indicator-match fusion
Use Case API (Vuln Chain, PKI Risk, Supply Chain Risk)	Domain endpoints for MVP use cases	REST	Stateless	**MVP** (Phase 1)

Component	Responsibility	Interface	Runtime	Phase
Use Case API (Zero-Day Triage, Blast Radius, Exposure)	Domain endpoints for Phase 2 use cases	REST	Stateless	**Phase 2**
Agent Supervisor	Orchestrate specialist agents; manage conversation/context	Internal	Stateless	**MVP** (Phase 1)
Specialist Agents (Topology, Vuln Chain, PKI, Evidence, Supply Chain Risk)	Bounded-scope agents for MVP use cases	Typed delegation contract	Stateless	**MVP** (Phase 1)
Specialist Agents (Zero-Day Triage, Ownership Resolution)	Bounded-scope agents for Phase 2 use cases	Typed delegation contract	Stateless	**Phase 2**
Eval Harness	Run gold-set evals, drift detection, regression alerts	Internal	Batch + on-demand	**MVP** (Phase 0)
Audit Service	Immutable, hashed audit log	Audit API	HA	**MVP** (Phase 0)
Kill Switch Service	Per-feature, per-agent, per-model, per-TI-source kill switches	Switch API	HA	**MVP** (Phase 0; per-TI-source switches added Phase 2)

---

6. Ontology & graph data model

The ontology is versioned, owned, and governed (§25). v1 covers MVP + immediate Phase 2 extensions for zero-day and supply chain.

Node types

Category	Node types
Assets	`Host`, `Container`, `Pod`, `Image`, `VM`, `NetworkDevice`, `LoadBalancer`, `APIGateway`
Applications	`BusinessApplication`, `Service`, `API`, `Microservice`
Identity	`User`, `ServiceAccount`, `AgentIdentity`, `Group`, `Role`
Credentials	`Certificate`, `Key`, `Secret`, `Token`
Vulnerabilities	`CVE`, `Finding`, `Weakness`, `Misconfiguration`
Exposure	`NetworkExposure`, `IdentityExposure`, `DataExposure`
Data	`Database`, `Table`, `Bucket`, `Topic`, `DataClassification`
Network	`NetworkZone`, `Segment`, `VPC`, `Subnet`

Category	Node types
Tenancy	`Tenant`, `BusinessUnit`, `LegalEntity`, `Region`
Ownership	`Team`, `Person`
Risk	`RiskScore`, `Control`, `ControlGap`
Evidence	`Document`, `Ticket`, `RunReport`, `ApprovalRecord`
Change	`Deployment`, `Patch`, `ConfigChange`
Threat (zero-day)	`Advisory`, `Indicator`, `ThreatActor`, `Campaign`, `TTP`, `ExploitKit`
Supply chain	`Package`, `PackageVersion`, `SBOM`, `BuildArtifact`, `BuildPipeline`, `CodeRepository`, `Commit`, `Vendor`, `ThirdParty`, `CodeSignature`, `Attestation`

Relationship types (selected)

Relationship	From → To	Cardinality	Key properties
`RUNS_ON`	`Service` → `Host|Container|Pod`	n:n	first_seen, last_seen
`DEPLOYED_TO`	`Image` → `Pod`	1:n	deployed_at, version
`USES_CERTIFICATE`	`Service|API|LoadBalancer` → `Certificate`	n:n	purpose
`HAS_VULNERABILITY`	`Asset` → `Finding`	n:n	first_seen, last_seen, status
`MANIFESTATION_OF`	`Finding` → `CVE`	n:1	confidence
`OWNED_BY`	`BusinessApplication|Asset` → `Team`	n:1	as_of, source
`HAS_ACCESS_TO`	`Identity` → `Asset|Database|Service`	n:n	permission, last_used
`EXPOSED_TO`	`Asset|Service` → `NetworkExposure`	n:n	confidence
`CONNECTS_TO`	`Service` → `Service|Database`	n:n	protocol, last_observed
`HAS_CLASSIFICATION`	`Database|Table|Bucket` → `DataClassification`	n:1	as_of
`DEPENDS_ON`	`Service` → `Service`	n:n	criticality
`LOCATED_IN`	`Asset` → `NetworkZone|VPC|Subnet`	n:1	source
`BELONGS_TO_TENANT`	(any) → `Tenant`	n:1	as_of
`REFERS_TO`	`Advisory` → `CVE|Package|TTP`	n:n	source_feed, source_trust
`PUBLISHES`	`Advisory` → `Indicator`	1:n	first_seen, decay_at
`MATCHES_INDICATOR`	`Asset|Image|Finding` → `Indicator`	n:n	confidence, matched_at, modality
`ATTRIBUTED_TO`	`Indicator|Campaign` → `ThreatActor`	n:n	confidence
`EMPLOYS_TTP`	`ThreatActor|Campaign` → `TTP`	n:n	source
`OBSERVED_IN`	`Indicator` → `Asset|Service`	n:n	first_seen, count, source

Relationship	From → To	Cardinality	Key properties
REQUIRES	PackageVersion|BuildArtifact → PackageVersion	n:n	scope (direct/transitive), version_constraint
CONTAINS	Image → PackageVersion	n:n	layer, source
BUILT_FROM	Image|BuildArtifact → BuildPipeline	n:1	build_id, built_at
SOURCED_FROM	BuildArtifact → CodeRepository	n:1	commit_sha
PUBLISHED_BY	PackageVersion → Vendor|ThirdParty	n:1	publisher_verified
SIGNED_BY	PackageVersion|Image|BuildArtifact → CodeSignature	n:1	algorithm, signature_verified, signed_at
ATTESTED_BY	BuildArtifact → Attestation	n:n	slsa_level, attestation_type
DESCRIBES	SBOM → Image|BuildArtifact	n:n	format (cyclonedx/spdx), generated_at
ACCESSES	ThirdParty|Vendor → Asset|Data|API	n:n	scope, last_used

Mandatory properties

All v2 properties hold (`id`, `source_system`, `source_record_id`, `tenant_id`, `classification`, `valid_from`/`valid_to`, `recorded_from`/`recorded_to`, `lineage_id`, `confidence_score`, `ontology_version`).

Additional for Indicators and Advisories:

Property	Purpose
`source_trust`	TI source trust score (0.0–1.0) at ingestion time
`decay_at`	When confidence should be re-evaluated or expired
`modality`	`cpe | hash | ip | domain | ttp | behavior | yara`
`severity_at_publish`	Severity asserted at publication

Additional for PackageVersions and BuildArtifacts:

Property	Purpose
`package_purl`	Package URL (PURL spec) — canonical identifier across ecosystems
`version`	Semantic version
`hash_sha256`	Content hash
`slsa_level`	0–4
`signature_verified`	Boolean + verification timestamp
`publisher_verified`	Boolean (against trusted publisher list)

Property	Purpose
`first_seen_in_env`	Earliest internal observation

Bitemporal — extended

Advisories and indicators get an additional **operational temporal axis**:

- `valid_from` / `valid_to`: when the advisory/IOC is asserted true in the world.
- `recorded_from` / `recorded_to`: when we knew about it.
- `actionable_from` / `actionable_to`: when triage decisions should be acted on (decay window).

This separation matters: an IOC may still be technically valid but no longer actionable because attacker infrastructure has moved.

Example: zero-day triage traversal

Given an advisory with CPE pattern matching, find affected production assets with exposure:

```
MATCH (a:Advisory {id: $advisoryId})-[:PUBLISHES]->(ind:Indicator)
MATCH (asset:Asset)-[:MATCHES_INDICATOR]->(ind)
WHERE asset.tenant_id IN $authorizedTenants
  AND asset.environment = 'prod'
  AND coalesce(ind.actionable_to, datetime() + duration('P30D')) >= datetime()
OPTIONAL MATCH (asset)<-[:RUNS_ON]-(svc:Service)<-[:CONTAINS_SERVICE]-(app:BusinessApplication)
OPTIONAL MATCH (svc)-[:CONNECTS_TO]->(db:Database)-[:HAS_CLASSIFICATION]->(dc:DataClassification)
OPTIONAL MATCH (e:NetworkExposure {type:'Internet'})-[:EXPOSED_TO]->(app)
RETURN asset, svc, app, dc.level AS data_class, e IS NOT NULL AS internet_exposed,
       ind.confidence AS indicator_confidence, ind.source_trust AS source_trust
ORDER BY internet_exposed DESC, data_class DESC, indicator_confidence DESC
LIMIT 500;
```

Example: supply chain transitive closure

Given a compromised PackageVersion, find all internal artifacts that contain it (transitively):

```
MATCH (compromised:PackageVersion {package_purl: $purl, version: $version})
CALL apoc.path.subgraphNodes(compromised, {
  relationshipFilter: '<REQUIRES|<CONTAINS',
  labelFilter: '+PackageVersion|+BuildArtifact|+Image',
  maxLevel: 12,
  bfs: true
})
YIELD node AS affected
WITH affected
MATCH (affected)<-[:DEPLOYED_TO|RUNS_ON*1..3]-(svc:Service)
WHERE svc.tenant_id IN $authorizedTenants
OPTIONAL MATCH (svc)<-[:CONTAINS_SERVICE]-(app:BusinessApplication)
OPTIONAL MATCH (app)-[:OWNED_BY]->(team:Team)
RETURN DISTINCT app.id AS app, team.name AS owner,
       svc.environment AS env, affected.package_purl AS path_pkg
LIMIT 1000;
```

Transitive closure traversal is gated by the **Transitive Closure Service** with explicit cost estimation; deep closures (depth > 12 or estimated rows > 10K) route async with reserved capacity.

---

7. Entity resolution architecture

ER governs identity matching across sources (the same asset seen in CMDB, Tanium, scanner). **Indicator matching is a distinct path** — see §7b — because the semantics differ (matching an *attribute pattern* against an entity, not matching two entity records).

Pipeline

```
Raw event → Canonicalize → Blocking → Candidate Match → Scoring →
Decision (auto-merge / queue / no-link) → Survivorship → Write →
Cascade Update → Audit
```

The full ER pipeline (blocking keys per entity type, deterministic vs probabilistic matching, tiered confidence decisions, survivorship matrix, merge/split cascade, weekly F1 eval against a gold set, and backpressure when the review queue exceeds threshold) was specified in v2 §7 and is unchanged in v3. The summary: blocking is mandatory (no pairwise comparison), deterministic matches on strong identifiers score 1.0, probabilistic matches use a calibrated GBM scorer monitored as a Tier 2 SR 11-7 model, decisions split into auto-merge ≥ 0.95 / review queue 0.70–0.95 / no-link < 0.70, and every link is bitemporal so retroactive corrections preserve audit trail.

7b. Indicator matching (distinct fuzzy path)

Aspect	Classic ER	Indicator Matching
Question	"Are these two records the same entity?"	"Does this entity match this pattern/signal?"
Input	Two records	One entity + one indicator
Output	Merge / split / link decision	Match confidence per (entity, indicator) pair
Frequency	High — every ingestion	High at advisory time, then steady
Decay	None	Indicators decay over time
Modalities	Identity attributes	CPE, hash, IP, domain, TTP behavior, YARA rule

Indicator match modalities

Modality	Match strategy	Store
CPE / package + version range	Range query over `PackageVersion.package_purl` + semver	Neo4j + Postgres index
File / image hash	Exact match	OpenSearch hash index
IP / domain	Exact + CIDR + suffix	OpenSearch
Behavior / TTP (MITRE ATT&CK)	Map TTP to detection rules in SIEM; lookup recent matches	SIEM bridge
YARA rule	Run rule on stored artifacts (limited; high cost)	Object store scanner (async)
Config pattern	Match config snapshots against advisory's preconditions	Postgres + Neo4j

Match scoring

Each (entity, indicator) match gets a confidence in [0,1]:

```
match_confidence =
  modality_baseline_confidence    # e.g., exact hash = 1.0; CPE range = 0.85; TTP = 0.6
  × source_trust(indicator)
  × decay_factor(now, indicator.published_at, indicator.half_life)
  × environment_specificity_adjustment
```

Source trust comes from the **TI Source Trust Registry**. Decay is per-modality (IPs decay fast — half life 7d; hashes decay slow — half life 180d).

Failure mode: IOC false-positive storm

A noisy feed can flood the platform with thousands of low-confidence matches. Controls:

- Per-feed circuit breaker on match rate.
- Source trust dynamically lowered when analyst feedback flags false positives.
- Match results below configurable confidence threshold (default 0.6) suppressed from analyst views but retained for audit.

---

8. Tenant isolation model

v2 §8 made the composite tenancy decision: a **single shared Neo4j database** holds the primary topology graph with mandatory tenant-predicate injection at the query planner and Neo4j role-based access as defense in depth, while **separate Neo4j databases** isolate two high-sensitivity scopes (privileged identity relationships and executive systems). Per-tenant DBs were rejected as a universal model because they would kill cross-tenant queries and make cross-tenant ER expensive. Enforcement is layered: API gateway authn + rate limit → Policy Decision Point resolves authorized tenants and classifications → Query Planner injects mandatory tenant predicate (query rejected if planner cannot prove tenant scope) → store-level controls as defense in depth → final result filter checks every emitted record's tenant_id against scope.

Zero-day burst: a new cross-tenant pattern

A zero-day advisory triggers **simultaneous queries across all tenants** to assess enterprise-wide exposure. This is a legitimate, predictable burst, not an exception path.

Design:

- **Pre-authorized burst mode**: Threat Intel Analyst role is pre-authorized for "advisory triage" cross-tenant queries against the approved Zero-Day Triage template library.
- **Burst rate limit**: Even authorized burst is rate-limited (default 10 advisories/hour per analyst) to prevent abuse.
- **Aggregated outputs first**: Default response is per-tenant counts; tenant-detail requires explicit per-tenant approval or formal incident ticket.
- **Separate audit stream**: Burst queries land in the cross-tenant audit stream with `burst_mode: true` and the advisory ID as correlation.

---

9. Data contracts framework

v2 §9 established that every source has a data contract registered in the Schema Registry and signed by the source owner — no source onboards without one. The contract specifies source identity and ownership, schema version, ontology mapping version, freshness SLA with on-breach action, completeness requirements (required fields, null tolerance per field), accuracy validation rules and reference checks, sensitivity classification per field, failure mode (degrade_with_warning / fail_closed / stale_allowed_with_warning), quarantine policy, and identity material with rotation cadence. Contract enforcement is automated: the Validation Service rejects non-conforming payloads to a quarantine topic, daily conformance reports run per source, and a source falling below 95% conformance for 7 consecutive days triggers an automatic ticket to the source owner plus degraded confidence in scoring.

v3 introduces two new contract archetypes:

Threat intel feed contract

```yaml
source: cisa_kev
owner_team: threat-intel
business_owner: jane.doe@bank
schema_version: 2.0.0
ontology_mapping_version: 1.0.0
source_trust_baseline: 0.95   # high trust; government source
feed_type: pull
freshness:
  sla: 30m
  on_breach: page on-call
indicator_modalities: [cpe, cve_reference]
decay_policy:
  default_half_life: 180d
  override_by_modality:
    ip: 7d
    domain: 14d
    hash: 180d
    cpe: 365d   # CPE-based indicators persist until vendor patches
quality_signals:
  false_positive_rate_monitor: enabled
  source_trust_recalibration: weekly
failure_mode: degrade_with_warning
quarantine_policy: "low-confidence or malformed indicators → quarantine"
```

SBOM source contract

```yaml
source: blackduck_sca
owner_team: app-security
schema_version: cyclonedx_1.5
ontology_mapping_version: 1.0.0
feed_type: push       # generated at build time
freshness:
  sla_per_image: 5m_post_build
  full_inventory_refresh: daily
completeness:
  required_fields: [image_hash, components, component_version, component_purl]
  null_tolerance_pct:
    component_purl: 0   # strict — without PURL, transitive closure breaks
accuracy:
  validation_rules:
    - "image_hash matches sha256 regex"
    - "every component has a purl"
    - "build_id resolves in build pipeline"
attestation:
  in_toto_required: true   # SLSA L3+
```

```
  signature_verification: required_for_prod
failure_mode: fail_closed   # missing SBOM → image not eligible for prod deploy (future enforcement)
quarantine_policy: "missing/malformed SBOM → quarantine + alert app owner"
```

---

10. Data flow & freshness tiers

Tier	Latency p95	Examples
T1 Hot	< 1 min	Known-asset vuln state change, cert expiry, **high-confidence IOC match on critical asset**
T2 Warm	< 15 min	New asset discovery, ownership re-resolution, SBOM ingestion, **advisory → indicator publication → environment match**
T3 Cold	< 1 hr	Risk score refresh, ontology migration, **transitive dependency closure recompute**, full SBOM reconciliation

Use case 4: Zero-Day Triage (T1 + T2 flow)

```
1. Advisory arrives via TI feed adapter (CISA, vendor PSIRT, internal IR)
2. Normalize to Advisory + Indicator nodes (modality-typed)
3. Apply source trust score from TI Source Trust Registry
4. Validate; quarantine malformed
5. Write Advisory + Indicators to Neo4j; index Indicators in OpenSearch (T1)
6. Indicator Matching Service runs sweep:
   - CPE/PURL match → Neo4j range query against PackageVersion + asset OS/software
   - Hash match → OpenSearch
   - IP/domain match → OpenSearch + SIEM bridge
   - TTP match → SIEM detection rule lookup
7. Each match yields a (entity, indicator, confidence) row, written as MATCHES_INDICATOR edge
8. Zero-Day Triage Scorer combines per-asset signals into a composite score:
     composite_score = probability_affected × topological_severity
   where probability_affected is the per-asset MATCHES_INDICATOR confidence
   (already includes source_trust and decay_factor per §7b),
   and topological_severity is derived from exposure, data classification,
   business criticality, and ownership context.
9. Topology context layered: exposure, data classification, ownership
10. Ranked exposure list emitted to Zero-Day Triage Console
11. Analyst triages, marks false-positives (feeds back to source trust + suppression)
12. Verified-affected assets create Findings (downstream goes through standard vuln chain)
```

The first 7 steps target < 5 min p95; the full ranked list to console targets < 15 min p95.

Use case 5: Supply Chain Risk (T2 + T3 flow)

```
1. Compromise advisory arrives ("npm package XYZ versions A-B are malicious")
```

```
2. Resolve to PackageVersion node(s) by package_purl + version constraint
3. Synchronous bounded closure (depth ≤ 8): "what internal artifacts directly or near-transitively contain this?"
4. If estimated closure exceeds the synchronous cap (depth > 8 or estimated rows > 10K — see §6), dispatch async deep closure (depth ≤ 12) on the reserved capacity pool
5. For each affected internal artifact:
   - Resolve to running services (via Image/Pod/Container deployment graph)
   - Apply runtime context: is it actually running? Where? Exposure?
   - Apply provenance: was build before/after compromise window? Signature valid?
6. Supply Chain Risk Scorer:
  risk = dependency_presence_confidence × runtime_observability × exposure × data_class × business_criticality
7. Generate impact subgraph; rank affected applications/services
8. Owner notifications + remediation suggestions (upgrade paths from package registry)
9. Audit trail with full closure provenance
```

Use case 3: Natural language hybrid query (extended)

Now includes indicator-match and transitive closure as routable executors:

```
"Are any of our prod apps affected by the xz-utils backdoor advisory?"

1. Policy: resolve scope
2. Intent Router: classify as hybrid (advisory lookup + transitive closure + topology)
3. Plan:
   ├── Lookup Advisory by name → Indicators
   ├── Transitive closure over REQUIRES + CONTAINS from xz-utils PackageVersion
   └── Graph: layer topology (services, apps, exposure, ownership, environment)
4. Parallel execute
5. Fuse: per-app impact summary with closure path as evidence
6. Final tenant + classification check
```

Context assembly (additions to the v2 context types)

Context type	Source	Refresh
TI source trust	TI Source Trust Registry	Per-query, cached 5m
Active advisories	Neo4j (filtered by `actionable_to >= now`)	Per-query
Recent IOC matches for user's tenants	OpenSearch	Per-query

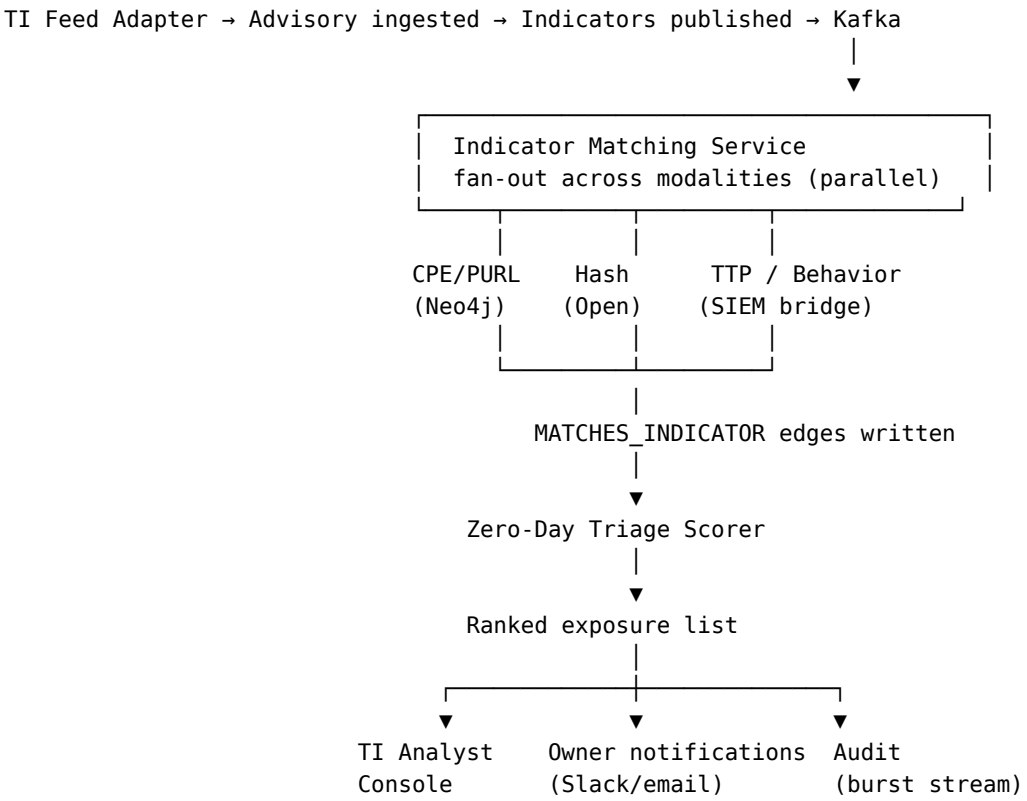
---

11. Sequence diagrams

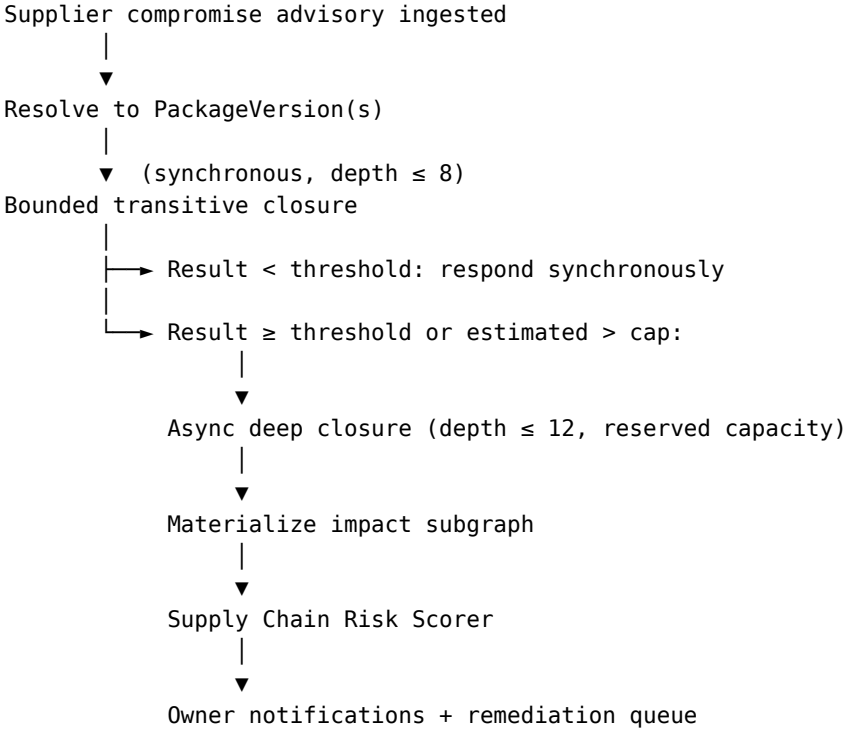
Happy path: hybrid topology query

The hybrid topology query path was diagrammed in v2 §11. Summary: request lands at API Gateway, routes through Policy → Intent Router → Query Planner, then the planner dispatches Graph / SQL / Vector subqueries **in parallel**, results are joined in the Result Fusion service, a final tenant + classification filter is applied, and the response goes back to the client. Parallel fan-out is what makes the p95 < 8s budget achievable: graph (2s budget) and SQL (3s budget) run concurrently, so the longer of the two dominates rather than summing. The end-to-end p95 budget is approximately 4.2s with parallel execution.

Zero-day triage burst



Supply chain compromise: transitive closure



Failure: TI feed becomes untrusted

```
Feed monitoring detects high false-positive rate from feed F
       │
       ▼
TI Source Trust Registry lowers F's trust score
       │
       ▼
Existing un-acted indicators from F: confidence recomputed
       │
       ▼
If confidence drops below threshold → suppress in analyst views, retain in audit
       │
       ▼
If feed F trust < 0.3 for 24h → kill switch on F (manual reactivation)
```

---

12. API & interface contracts

Common headers (all APIs)

Every API request carries `Authorization: Bearer <token>` (OIDC user or actor token), `X-Correlation-ID` (end-to-end correlation across services), `X-Trace-ID` (distributed tracing), `X-Requesting-Agent-ID` (present when an agent acts on behalf of a user), `X-Execution-Budget-Ms` (max wall time the client will wait), `X-Max-Cost-Units` (cost cap for backend execution — graph hops, SQL bytes, model tokens), and `X-Idempotency-Key` (required for state-changing operations).

Zero-day triage API

```
POST /v1/zero-day/triage
Authorization: Bearer ...
X-Correlation-ID: ...

{
  "advisory_id": "ADV-2026-1042",
  "scope": {
    "tenants": ["consumer-banking", "wealth-mgmt"],
    "environments": ["prod"]
  },
  "response_preferences": {
    "include_evidence": true,
    "max_results": 500
  }
}
```

Response:

```
{
  "correlation_id": "...",
  "advisory": {
    "id": "ADV-2026-1042",
    "title": "Pre-auth RCE in libfoo",
    "source": "vendor-psirt",
    "source_trust": 0.95,
    "published_at": "2026-05-12T08:00:00Z",
    "indicators_count": 4
  },
  "exposure": {
    "total_assets_matched": 142,
    "by_tenant": {"consumer-banking": 89, "wealth-mgmt": 53},
```

```
    "by_environment": {"prod": 142},
    "by_confidence_bucket": {"high": 24, "medium": 71, "low": 47}
  },
  "ranked_findings": [
    {
      "rank": 1,
      "asset_id": "...",
      "application_id": "...",
      "owner_team": "...",
      "probability_affected": 0.91,
      "topological_severity": 0.88,
      "composite_score": 0.80,
      "internet_exposed": true,
      "data_classification": "restricted",
      "evidence_refs": ["..."],
      "lineage_id": "..."
    }
  ],
  "warnings": [],
  "execution": {"duration_ms": 8200, "modalities_matched": ["cpe", "hash"]}
}
```

Supply chain impact API

```
POST /v1/supply-chain/impact
Authorization: Bearer ...

{
  "package": {
    "purl": "pkg:npm/event-stream",
    "version_constraint": ">=3.3.6,<3.3.7"
  },
  "scope": {"tenants": ["consumer-banking"], "environments": ["prod"]},
  "include_transitive": true,
  "max_depth": 12,
  "async_acceptable": true
}
```

Response (synchronous when bounded; async via callback or polling for deep closures):

```
{
  "correlation_id": "...",
  "closure": {
    "depth_traversed": 9,
    "internal_artifacts_affected": 38,
    "images_affected": 12,
    "running_services_affected": 8,
    "applications_affected": 3,
    "computed_at": "..."
  },
  "ranked_applications": [
    {
      "rank": 1,
      "application_id": "...",
      "owner_team": "...",
      "running_services": [...],
      "dependency_paths": [
        ["pkg:npm/event-stream@3.3.6", "pkg:npm/flatmap-stream@0.1.1",
         "internal-lib-streams@2.4.0", "image:customer-api:v8.3.1"]
      ],
      "build_provenance": {
        "slsa_level": 2,
        "signature_verified": true,
        "built_before_compromise_window": false
      },
      "supply_chain_risk_score": 87,
```

```
      "evidence_refs": ["..."]
    }
  ],
  "remediation_suggestions": [
    {"action": "upgrade", "package": "event-stream", "to_version": "4.0.1",
     "available_in_registry": true}
  ]
}
```

Error taxonomy additions

Code	Meaning	Recovery hint
`TI_SOURCE_KILLED`	Feed is on kill switch	Use alternate feed or wait for re-enable
`CLOSURE_TOO_DEEP`	Transitive closure exceeded depth cap	Use async mode or narrow scope
`CLOSURE_TOO_EXPENSIVE`	Estimated rows above cap	Use async mode
`ADVISORY_NOT_FOUND`	Advisory ID unknown	Wait for ingestion or use external reference
`SBOM_MISSING`	No SBOM for requested image	Trigger SBOM generation or use manual inventory
`ATTESTATION_INVALID`	Signature or attestation verification failed	Investigate; do not deploy

---

13. Data model & storage strategy

v2 §13 specified the core polyglot stores: Neo4j Enterprise for topology relationships, BigQuery and Postgres for structured analytics and platform metadata, an approved vector store for semantic context, OpenSearch for text and log search, an object store for evidence and replay, Kafka for events, Redis for session state, and an append-only audit log replicated to immutable object storage. The additions below extend that with stores specific to indicators, package dependencies, SBOMs, attestations, and the materialized closure projection.

Dataset	Store	Why
Indicators (active)	Neo4j + OpenSearch	Graph for relationships to advisories/TTPs; OpenSearch for fast IOC index by hash/IP/domain
Indicator match observations	Neo4j (edges) + BigQuery (analytics)	Graph for traversal; BigQuery for trend/effectiveness analytics
PackageVersions + dependencies	Neo4j	Native graph traversal for transitive closure
SBOM raw	Object store	Replay, audit, lineage; mirror of source-of-truth
Transitive closure materialization	Postgres + Neo4j projection	Pre-computed closures for hot packages refreshed nightly + on-change

Dataset	Store	Why
Attestations & signatures	Postgres + Neo4j refs	Verification record + edge to artifact

Indexing additions

```
CREATE INDEX indicator_modality FOR (n:Indicator) ON (n.modality, n.value_hash);
CREATE INDEX packageversion_purl FOR (n:PackageVersion) ON (n.package_purl, n.version);
CREATE INDEX advisory_published FOR (n:Advisory) ON (n.published_at);
CREATE INDEX image_hash FOR (n:Image) ON (n.hash_sha256);
CREATE INDEX builds_in_window FOR (n:BuildArtifact) ON (n.built_at);
```

OpenSearch indexes:

```
indicator_hash_idx       (hash → indicator_id, advisory_id, confidence)
indicator_network_idx    (ip / domain / cidr → indicator_id)
indicator_cpe_idx        (cpe → indicator_id, version_constraint)
```

Transitive closure pre-computation

For high-fanout packages (top 1% by usage), nightly pre-computation of the closure subgraph into a materialized projection. Synchronous queries against materialized closures return in < 1s; on-demand closures for cold packages fall through to live traversal with capacity reservation.

---

14. Auth & authorization pattern

The authentication and authorization model from v2 §14 is unchanged in v3. Summary: human users authenticate via enterprise SSO/OIDC; service accounts use mTLS with workload identity (SPIFFE); agents have their own OIDC identities. Authorization is layered — RBAC (roles like `security_analyst`, `pki_team`, `cyber_leadership`), ABAC (attributes including `authorized_tenants` and `authorized_classifications`), ReBAC (relationship-based — "is this user on the team that owns this app?"), and tool-level authorization for agents (each agent has a fixed tool list; tool calls authorize independently of agent identity). When an agent acts on behalf of a user, the API gateway performs OAuth 2.0 Token Exchange (**RFC 8693**) to produce an actor token where `subject = user`, `actor = agent_id`, `scope = user_scope ∩ agent_max_scope`. Audit records both subject and actor for every action.

New roles

Role	Pre-authorized scope
`threat_intel_analyst`	Cross-tenant burst mode for Zero-Day Triage approved templates; aggregate-only by default; per-tenant detail requires linked incident ticket
`software_supply_chain_owner`	Cross-tenant aggregates over PackageVersion/SBOM data; tenant-scoped detail

---

15. Agent runtime architecture

The agent runtime topology from v2 §15 is unchanged in v3. Summary: a single **Agent Supervisor** orchestrates **Specialist Agents** via a typed (protobuf-style) delegation contract. Specialists do not call each other directly. The supervisor holds the user-facing conversation and the running plan; specialists receive only task spec + retrieved context + tool results, never the full conversation. Inter-agent results above 2k tokens are passed by reference (evidence pointers), not embedded. Any specialist that hits its budget, encounters a tool error, or detects ambiguity returns with a status that surfaces to supervisor; supervisor decides whether to retry, route to alternate agent, escalate to HITL, or return partial answer with warnings. Specialists never silently retry.

Capability matrix — extended

Agent	Tools allowed	Max steps	Max cost	Can delegate?	Write access	HITL required for	Eval suite	SR 11-7 tier
Supervisor	Specialist delegation API, Intent Router, Policy PDP	30	10,000	yes (specialists only)	none	cross-tenant queries (excl. pre-authorized burst), high-cost queries, ambiguous intent	supervisor-eval-v1	2
Topology Analyst	Graph Query Svc, Vector RAG	8	1,500	no	none	none in MVP	topo-eval-v1	2
Vulnerability Chain	Graph Query Svc, NL2SQL, Risk Scoring, Vector RAG	10	2,500	no	none	when ranking influences SLA	vuln-eval-v1	1
PKI Intelligence	Graph Query Svc, NL2SQL	8	1,500	no	none	proposing revocation candidates	pki-eval-v1	2

Agent	Tools allowed	Max steps	Max cost	Can delegate?	Write access	HITL required for	Eval suite	SR 11-7 tier
Ownership Resolution	Graph Query Svc, SQL Gateway	6	1,000	no	none	proposing new owners	ownership-eval-v1	2
Evidence	Vector RAG, OpenSearch, Object Store read	6	1,000	no	none	none	evidence-eval-v1	3
Zero-Day Triage	Indicator Match Svc, Graph Query Svc, Zero-Day Scorer, Vector RAG, TI Source Trust Registry	12	3,000	no	none (writes triage findings via use case API only)	declaring an asset affected when source trust < 0.7; cross-tenant escalation; suppression of indicators	zeroday-eval-v1	**1**
Supply Chain Risk	Transitive Closure Svc, Graph Query Svc, NL2SQL, Provenance Verifier, Supply Chain Risk Scorer	14	4,000 (closure-heavy)	no	none	proposing image rebuild orders; flagging vendor compromise; closures over depth 12	supplychain-eval-v1	**1**

Both new agents are **Tier 1** because their outputs directly inform incident response decisions and remediation prioritization at enterprise scale. Validation, monitoring, and approval gates are correspondingly stricter.

16. Governed query generation: SQL and Cypher

The platform exposes two query-generation paths: natural language to SQL (for structured analytical data in BigQuery and Postgres) and natural language to Cypher (for topology and relationship traversal in Neo4j). Both use constrained LLM generation, both pass through a gateway that enforces governance, and both are symmetric in their controls. Templates are preferred over generation whenever the question matches a registered signature.

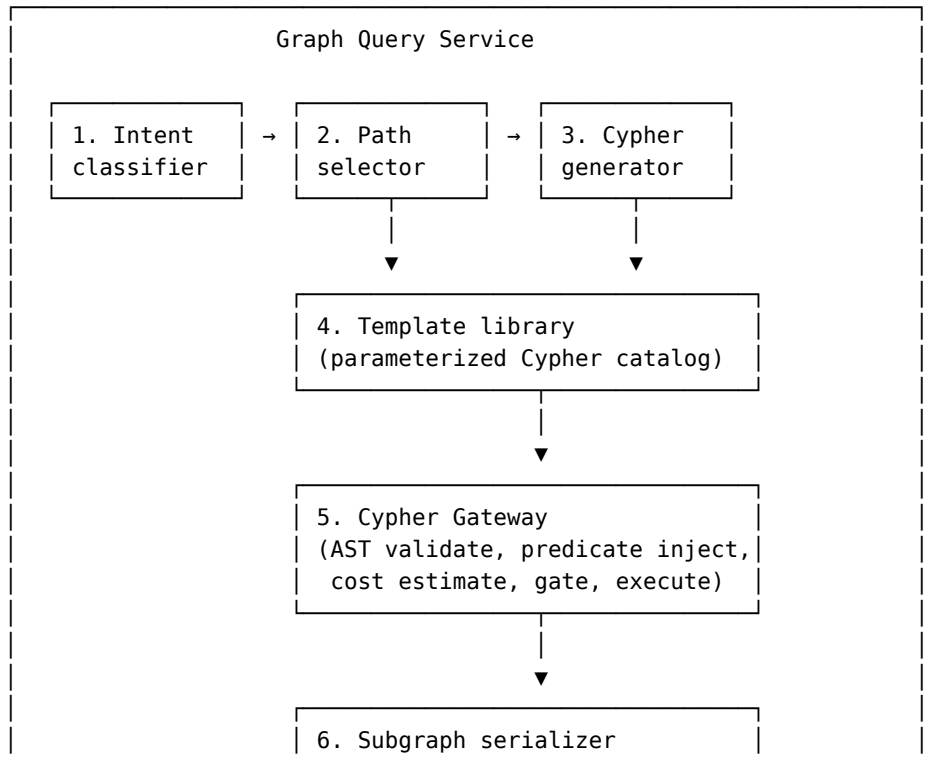
16.1 NL2SQL governance

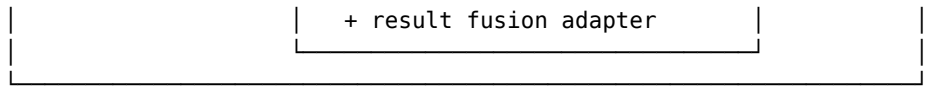
NL2SQL governance from v2 §16 is unchanged in v3. Summary: question → intent classifier → schema retrieval (vector search over column descriptions, constrained to user's authorized scope) → constrained generation with schema-aware prompt at low temperature → SQL AST parse → AST validator enforcing read-only, mandatory tenant predicate injection, banned constructs (e.g., recursive CTEs above depth 5, cross joins without limits), column allowlists per table, and row limits → cost estimator (route to HITL above threshold) → execution via the SQL Gateway with statement timeout and row cap → column masking by classification + final tenant filter on results. The eval harness measures execution accuracy, schema linking precision, predicate correctness, tenant leakage (gated at 0), and prompt injection resistance (gated at ≥ 99%) — nightly plus on every model or template change.

The schema scope for v3 adds the following tables to the approved NL2SQL surface: `advisory_summary`, `indicator_match_observations`, `sbom_components`, `package_inventory`.

16.2 Graph Query Service: internal architecture

The Graph Query Service is the component that agents and use case APIs call when they need graph data. From the caller's perspective it is one tool with a single API. Internally it decomposes the request through six deterministic stages — **not** internal subagents. Subagents are explicitly rejected: determinism is the security property, the LLM is constrained to one stage, and the audit trail must stay attributable to specific stages rather than nested model decisions.





Stage 1 — Intent classifier. Deterministic classifier (rules + small classifier model) that maps the incoming request to one of: `typed_intent` (from a use case API), `parameterized_template` (matches a registered template signature), `natural_language` (free-form question), `graphrag_request` (anchor-and-expand pattern, Phase 2/3). No LLM. Latency budget < 50ms.

Stage 2 — Path selector. Decision table in code that picks the execution path based on the intent type:

Intent	Path	Rationale
Typed call from use case API	Template (parameterized)	Already validated at registration, fixed cost
Typed delegation from agent	Template (parameterized)	Same as above
NL matching a known template signature	Template (parameterized)	Skip generation when we already have a safe query
NL not matching any signature	NL2Cypher generation (stage 3)	Free-form path; the exception, not the rule
Anchor-and-expand question	GraphRAG (Phase 2/3)	Embedding-based retrieval + structural expansion

Decision table lives in code, not in a prompt. No LLM. Latency budget < 50ms.

Stage 3 — Cypher generator (only on the NL2Cypher path). Schema-aware constrained generation:

- Retrieves graph schema (node labels, relationship types, property names) **scope-filtered to authorized labels** via the policy decision point.
- Generates **parameterized Cypher** with named parameters; never inline literals.
- Low temperature, structured output.
- Validates the AST locally before passing to gateway; regenerate-once-with-hints on failure.
- Returns: Cypher string + named parameters + confidence + node/edge labels referenced.

Same governance posture as NL2SQL: confidence score with regenerate-once-on-failure, HITL escalation on persistent failure or low confidence. This is the only stage that invokes an LLM. Latency budget 200–800ms depending on model.

Stage 4 — Template library. The catalog of pre-written parameterized Cyphers. Each template is:

- Named (`vuln_chain_for_asset`, `pki_downstream_impact`, `transitive_dep_closure_bounded`, `blast_radius_from_ass` `ownership_path`, `cert_dependency_reach`, …).
- Versioned and registered in the Use Case Registry (§29.3) alongside the use case it serves.
- **Already validated by the Cypher Gateway at registration time** — predicate placeholders, depth caps, LIMIT placeholders, banned-construct check all pass.
- Bound to a specific use case API or agent (capability containment — the Topology Analyst doesn't get templates registered to the Vuln Chain Agent).
- Has a documented cost envelope from registration (typical row count at typical parameter values).

Templates are the **dominant path**: typically 90%+ of traffic. NL2Cypher is the exception for free-form questions. This ratio is deliberate — registered, validated, cost-bounded queries are always preferred over freshly-generated ones.

Stage 5 — Cypher Gateway. The load-bearing control. Symmetric to the SQL Gateway from §16.1:

- AST parse the Cypher (whether from a template or NL2Cypher).
- Validate:
  - Read-only enforcement — reject any write, schema change, or APOC procedure with side effects.
  - Mandatory `tenant_id` predicate — every matched node must have a tenant predicate; planner injects if permissible, otherwise reject. Templates have this baked in; NL2Cypher output gets it injected at validation.
  - Banned constructs — no unbounded `*` paths, no Cartesian products on large labels, no `apoc.cypher.run(<dynamic string>)`, no recursive `*..*` without explicit depth cap, no non-allowlisted APOC procedures.
  - Depth cap enforcement — template's documented depth or default cap (8 for sync, 12 for async per §17).
  - `LIMIT` injection — default 10,000 paths; up to 100,000 with HITL approval.
- Cost estimator — Neo4j `PROFILE` or `EXPLAIN` against a small sampled slice; estimate row count, expected memory, expected time.
- Gate decision: `ok` → proceed; `expensive` → HITL approval flow; `unsafe` → reject 4xx.
- Execute against Neo4j with statement timeout (default 30s, configurable per template).
- Post-process: classification masking on returned node properties (`customer_ssn`, `pii_email`), final tenant filter on returned nodes as a row-level safety net even though the planner injected a predicate.

Every Cypher string that reaches Neo4j goes through here regardless of origin. There is no path around it. This is symmetric to the SQL Gateway and applies identically to template-driven and NL2Cypher-driven queries.

Stage 6 — Subgraph serializer + result fusion adapter. Shapes the Neo4j result for the caller. Three output modes:

- **Result Fusion mode** (when called from Hybrid Query) — structured records keyed by entity ID, ready to merge with SQL and vector results at the Result Fusion service.
- **Agent response mode** (when called via typed delegation) — typed protobuf-style result with evidence references (node IDs, path IDs) rather than inline content if the result is large (per §15 evidence-by-reference above 2k tokens).
- **GraphRAG mode** (Phase 2/3) — serialized subgraph (paths + properties + relationships) suitable to embed as LLM context for downstream reasoning, with citations preserved.

16.3 Worked examples

Template path (the common case). A user asks via Hybrid Query: "Show me the blast radius of asset X within 4 hops."

1. Intent classifier: topology question with explicit hop depth.
2. Path selector: matches `blast_radius_from_asset(asset_id, depth)` template signature. No generation.
3. Template library returns parameterized Cypher with `$asset_id` and `$depth` placeholders.
4. Cypher Gateway: parse, validate (predicate baked in at registration time), cost estimator returns ~3K rows for depth=4 on typical asset. Gate passes.
5. Execute against Neo4j.
6. Subgraph serializer shapes the paths into Result Fusion format.

Internal latency: ~50ms (no LLM call) plus Neo4j execution.

NL2Cypher path (the exception). Topology Analyst agent receives: "How does this app reach restricted data, going through certificates that are expiring soon?"

1. Intent classifier: free-form natural language, multi-concept.
2. Path selector: no template signature match; route to NL2Cypher.

3. Cypher generator retrieves graph schema scope-filtered to authorized labels (Asset, Service, BusinessApplication, DataAsset, Certificate, relationships HOSTS, RUNS_ON, USES, …). Generates parameterized Cypher with low temperature.
4. Cypher Gateway validates. If validation fails (e.g., hallucinated relationship type), regenerate once with hints; if it fails again, HITL.
5. Cost estimator runs PROFILE.
6. Execute (or HITL).
7. Subgraph serializer returns a typed result.

Internal latency: ~500–1500ms for the generation step plus execution. This path is the exception; templates carry the bulk of traffic.

16.4 Cypher Gateway governance and eval

Same eval cadence and gates as NL2SQL (§16.1):

Metric	Threshold	Frequency	Gate type
Cypher execution accuracy vs gold set	≥ 0.85	Nightly + on model/template change	Gate
Schema linking precision (correct labels referenced)	≥ 0.95	Nightly	Gate
Mandatory tenant predicate present	100%	On every executed query	Gate (reject)
Tenant boundary adversarial suite	100% pass	Hourly	Gate
Banned construct rejection rate	100%	On every rejected query	Audit
Cost estimator calibration (estimate vs actual)	±25%	Weekly	Alert
HITL approval rate	< 5%	Weekly	Alert if higher

Cypher Gateway has the same kill-switch posture as SQL Gateway: kill switch per template, per agent tool allowlist, per generated-Cypher pattern. A regression in NL2Cypher can be contained without affecting template execution.

16.5 GraphRAG (Phase 2/3 forward-looking)

GraphRAG is a distinct retrieval pattern from "agent calls Cypher then reasons over the result." The pattern:

1. Embedding lookup against **node and edge embeddings** to find anchor nodes for the question.
2. Structural expansion from anchors — k-hop neighborhood, shortest path between anchors, community detection.
3. Subgraph serialization (paths, nodes-with-properties, edges).
4. LLM reasons over the serialized subgraph as context.
5. Answer with citations to nodes and edges as evidence.

This is different from Vector RAG (which retrieves text chunks) because the retrieval unit is a **subgraph**, and from a Cypher-writing agent because the retrieval is *learned* (embeddings find

anchors, structural rules expand). The agent never writes Cypher; it works with the assembled subgraph.

Scope and status:

- GraphRAG is **Phase 2 or Phase 3**, not MVP. It requires node and edge embeddings (the Embedding Pipeline component is text-only in MVP — extending it to graph embeddings is non-trivial work) and a defined retrieval-quality eval framework.
- The Subgraph serializer (stage 6 of Graph Query Service) already produces a GraphRAG-compatible output mode, so the integration point exists; the missing pieces are the embedding model, the retrieval logic, and the eval suite.
- Eval question to resolve before scoping: what does "Recall@k for relevant subgraph" mean operationally? Is the gold standard a human-curated subgraph per question, or sampled paths with binary relevance labels? This needs a research spike in Phase 2.

For MVP and Phase 1, the Topology Analyst agent uses the template + NL2Cypher paths (stages 4 and 3) for free-form topology questions. GraphRAG replaces the NL2Cypher path for a specific class of question — "find the relevant neighborhood for this concept" — when it becomes ready.

---

17. Non-functional requirements

The non-functional requirements from v2 §17 apply: 99.9% monthly availability for query APIs; 99.5% for ingestion pipelines; RTO 30 min (API), 4 hr (ingestion), 8 hr (full graph restore); RPO 15 min for metadata and 0 for Kafka events (durable + replayable); hybrid query p95 < 8s (target 5s); graph-only < 3s; SQL-only < 5s; cached dashboards < 1s p95; ingestion freshness per the three tiers (T1 < 1 min, T2 < 15 min, T3 < 1 hr); initial throughput 100 query req/min scaling horizontally to 1000+; Phase 1 graph scale 10M nodes / 30M edges, Phase 3 100M nodes / 500M edges; 100% of queries through the SQL Gateway, all read-only, all tenant-predicate-enforced; 100% pass on adversarial tenant-boundary suite (hourly); 100% of query/agent/policy decisions audited, hash-chained, 7-year retention; per-query and per-tenant cost caps; pinned model versions with 90-day deprecation and canary 1% → 10% → 100% rollout.

The v3 additions:

Category	Requirement
Zero-day triage latency	Advisory ingestion → ranked exposure list < 15 min p95
Zero-day burst capacity	Support 5 concurrent advisory triages per tenant cluster
Supply chain closure latency	Bounded closure (depth ≤ 8, top 99% packages) < 30s p95; deep closure async with reserved capacity
SBOM ingestion latency	< 5 min from build completion to SBOM in Neo4j
TI source quarantine response	Auto-suppress within 1 minute of source-trust drop
Indicator index freshness	New indicators searchable within 30s of ingestion

---

18. Failure modes & recovery

v2 §18 specified failure modes for source connector failure, bad schema mapping, ER quality regression, Neo4j unavailability, vector DB unavailability, SQL source timeout, policy service down, agent infinite loop, context poisoning, embedding drift, ontology version mismatch, query composition attacks, prompt injection via ingested documents, source connector compromise, privileged operator insider threat, model regression, and cost runaway. Each carries a defined blast radius, detection signal, containment action, recovery procedure, and named owner. The v3 additions below extend that table with threat-intel and supply-chain specific modes.

Failure mode	Blast radius	Detection	Containment	Recovery	Owner
TI feed compromised or poisoned	Bad indicators trigger false-positive triage cascade	Source trust monitor, anomalous match rate	Kill switch on feed; suppress matches from feed	Re-validate feed; clean affected match edges; relabel sample of triages	TI team
IOC false-positive storm	Analyst fatigue; SOC overload	Match rate spike per feed	Per-feed circuit breaker on match rate	Lower source trust; tighten suppression threshold; review with feed vendor	TI team
Advisory de-duplication failure	Same advisory ingested multiple times from multiple feeds creates duplicate triages	Advisory cluster anomaly	De-dup pipeline review	Merge advisory nodes; relink edges via bitemporal supersede	TI team
TTP/behavior match latency	SIEM bridge slow; behavior-based matches lag	Latency per modality	Fall back to other modalities; warn on partial result	Investigate SIEM connector	Platform team
SBOM missing or stale	Image deploys without provenance; transitive closure incomplete	SBOM coverage metric; conformance < 95%	Block prod deploy (Phase 3); warn in Phase 1; analyst dashboard flag	Regenerate SBOM; backfill	App security team
Transitive dep explosion	Closure query OOMs or times out	Cost estimate at planner	Force async mode; depth cap; return partial with warning	Pre-materialize closure for the affected package	Platform team

Failure mode	Blast radius	Detection	Containment	Recovery	Owner
Signature/attestation verification bypass	Compromised artifact passes provenance check	Audit anomaly, divergence between attested and observed	Re-verify; quarantine artifact lineage	Investigate attestation chain; revoke signing key if needed	App security + security ops
Package registry compromise	All packages from registry suspect	TI advisory; vendor notification	Quarantine all packages ingested in window; re-verify signatures	Re-fetch from clean source; rebuild affected artifacts	App security + platform
Closure pre-materialization drift	Materialized closures stale vs live graph	Hash divergence between materialized and live	Force live traversal; rebuild materialization	Schedule incremental refresh; tighten triggers	Platform team
Vendor risk feed lag	Third-party access exposure stale	Freshness metric on vendor inventory	Warn on stale data; degrade vendor risk score confidence	Re-pull TPRM	App security + vendor mgmt

Mandatory circuit breakers (additions)

- Max concurrent indicator-match jobs per tenant
- Max transitive closure depth (default 12; async reserved beyond)
- Max materialized closure age (default 24h; force refresh)
- Per-feed indicator ingestion rate cap
- Kill switch per TI feed
- Kill switch per SBOM source

---

19. Observability & evaluation

v2 §19 specified the observability signals: logs (connector events, normalization failures, ER decisions with feature contributions, graph writes, generated SQL/Cypher with model version, policy decisions, agent tool calls with budgets); metrics (ingestion lag per source, ER confidence distribution, query latency per service, denial rate, partial result rate, agent steps, cost per query, model quality KPIs); traces (gateway → policy → router → planner → executor(s) → fusion → API, with span attributes for model versions and tenants); and evals (intent routing, NL2SQL execution accuracy, graph path precision, tenant filtering, answer faithfulness, ER F1, RAG nDCG). Drift detection has a 24-hour SLA for most signals, hourly for tenant boundary tests, weekly for ER F1 and embedding nDCG. The v3 additions below introduce eval suites for the new use cases.

New eval suites

Suite	Metrics	Frequency	Gating
`zeroday-eval-v1`	Triage precision@20, triage recall on labeled advisories, latency to ranked list	Weekly on gold set; on every model/template change	Precision@20 ≥ 0.85; latency < 15 min p95
`supplychain-eval-v1`	Closure correctness vs ground truth (sampled), impact ranking precision@10, missing-SBOM rate	Weekly; on every change	Closure precision ≥ 0.95; ranking precision@10 ≥ 0.85
Indicator match	Modality-specific precision/recall on labeled set, decay calibration	Weekly	Precision ≥ thresholds per modality; calibration error < 5%
TI source trust calibration	Per-feed FP rate vs declared trust	Weekly	Calibration drift triggers retrain of trust model

Drift detection — additions

- Indicator match precision: 2pp drop → sev-2.
- Supply chain closure precision: 1pp drop → sev-1 (incorrect impact answers are operationally severe).
- TI source trust calibration: KS-test against baseline weekly.

---

20. Security, compliance & model risk

v2 §20 specified the STRIDE threat model (spoofing → OIDC/mTLS/SPIFFE/token exchange; tampering → signed ingestion, schema validation, bitemporal supersede, hash-chained audit; repudiation → immutable audit + export workflow; information disclosure → defense-in-depth ABAC/query rewriting/store-level access/result filter/adversarial tests; denial of service → cost caps, traversal depth caps, query timeouts, circuit breakers, kill switches; elevation of privilege → tool-level authorization, signed tool registry, enforced agent capability matrix). It also specified the security controls (TLS 1.3 + mTLS in transit, AES-256 at rest with HSM keys, Vault-managed secrets rotated per policy, OpenShift network policies, SPIFFE workload identity, classification-based data masking, SQL/Cypher AST validators, SBOM per build, hash-chained 7-year audit). The model risk register established eight models with SR 11-7 tiering: intent classifier (Tier 3), embedding model (Tier 3), NL2SQL generator (Tier 2 analyst / Tier 1 decision-support), ER match scorer (Tier 2), risk scoring model (Tier 1), agent planner LLM (Tier 2), answer faithfulness judge (Tier 3, monitoring only), Cypher generator if/when added (Tier 1). The v3 additions below extend the model register and STRIDE table.

Model risk — additions

Model	Purpose	Tier	Validation	Owner	Approval gate
Zero-Day Triage scorer	Combine probability_affected (which already encapsulates source_trust × decay) with topological_severity to produce a per-asset composite score	**1**	Precision@K on labeled advisories; calibration; explainability of feature contributions	TI team	MRM committee + IR sponsor
Supply Chain Risk scorer	Combine dependency presence × runtime exposure × data class × business criticality	**1**	Precision@K; reproducibility; alignment with SME ranking	App security team	MRM committee + cyber sponsor
Indicator match confidence calibrator	Calibrate per-modality match confidence	**2**	Calibration error, modality-specific precision	TI team	MRM committee
TI Source Trust model	Per-feed trust score from FP feedback + source priors	**2**	Calibration vs analyst-labeled FP rate	TI team	MRM committee
Indicator decay model	Time-decay of indicator confidence per modality	**3**	Empirical decay vs theoretical	TI team	Tech lead

Threat model additions (STRIDE)

Threat	Example	Control
TI poisoning	Adversary submits indicators to a trusted feed to map our environment	Source trust scoring, anomaly detection on feed delta, kill switch, no automated remediation actions in MVP
Supply chain attestation forgery	Forged in-toto attestation for a malicious build	Multi-party attestation verification; signing key rotation; verify against trusted root
Closure DoS	Adversary triggers expensive transitive closure to exhaust resources	Cost estimate gate; async reservation pool with per-tenant quotas; rate limit

---

21. Cross-tenant query handling

v2 §21 established that cross-tenant access is a first-class, gated path, not an exception. Authorized roles include `cyber_leadership` (read-only aggregates), `auditor` (read-only with evidence access scoped to audit window), and `cyber_global_analyst` (cross-tenant read for IR with per-incident scope). A library of approved cross-tenant templates runs autonomously for authorized roles; anything not in the library routes to HITL. Default mode is aggregation only; detail-level cross-tenant access requires named justification linked to a ticket, per-request approval, time-boxed access, and writes to a separate audit stream with mandatory after-action review. Burst mode handling for zero-day is detailed in §8.

Approved cross-tenant template library — additions

- "Zero-day exposure count by tenant for advisory X"
- "Top 10 most-affected tenants for advisory X" (aggregate only)
- "Packages affected by SBOM advisory X by tenant" (aggregate)
- "Vendor compromise impact by tenant"

Tenant-detail beyond aggregate requires:

- Open incident ticket
- Per-tenant approval from tenant security lead
- Time-boxed access (default 4h)
- Separate cross-tenant burst audit stream

---

22. MVP scope

Hypothesis (extended)

If CMDB + Vuln Scanner + PKI + Ownership + **SBOM** data are connected in a tenant-aware cyber knowledge graph with governed query orchestration, security analysts can prioritize remediation faster, answer supply chain reach questions in seconds instead of days, and have a foundation for zero-day triage in Phase 2.

In scope (Phase 1 MVP)

Capability	Detail
Data sources	ServiceNow CMDB, Qualys (or Tenable), Venafi (or DigiCert), ownership mapping, **Black Duck SCA (SBOM source)**, **container registry**
Stores	Neo4j (shared primary), Postgres (metadata), BigQuery (analytics), approved vector store, OpenSearch, object store
Ontology	v1 covering Asset, Application, Service, Certificate, Vulnerability, Finding, User, Team, Tenant, DataClassification + Package, PackageVersion, SBOM, Image, BuildArtifact

Capability	Detail
ER	Full pipeline with blocking, scoring, survivorship, review queue
Tenancy	Composite model with mandatory predicate injection; adversarial test suite
Agents	Supervisor + Topology + Vulnerability Chain + PKI + Evidence + **Supply Chain Risk**
Use cases	Vulnerability Chain, PKI Risk, **Supply Chain Risk (bounded)**
Transitive closure	Bounded synchronous (depth ≤ 8) for top 1% pre-materialized packages; async deep closure scaffolded but limited
NL2SQL	Read-only against approved tables
Vector RAG	Runbooks, schema docs, policy docs
UI	Search, topology view, vuln chain dashboard, PKI dashboard, **supply chain dashboard**, evidence panel
APIs	Hybrid query, graph query, SQL execute, PKI risk, vuln chain, **supply chain impact**
Audit	Full hash-chained logging
Deployment	OpenShift, single region

Explicitly NOT in MVP (deferred to Phase 2)

- **Zero-Day Triage use case** (requires TI infrastructure)
- TI feed ingestion (other than CVE feed for context)
- IOC matching service (scaffolded only)
- Behavioral/TTP matching
- Full transitive closure pre-materialization for all packages
- Build provenance enforcement (signature verification at deploy gate)
- Vendor/third-party risk integration

MVP services (Phase 0 + Phase 1) at a glance

The components below are the complete service set required to ship MVP. Anything not on this list is deferred.

Foundation services (Phase 0):

- Event Bus (Kafka)
- Ontology Registry · Schema Registry · Model Registry
- TI Source Trust Registry (scaffolded, no live feeds)
- Policy Service (PDP)
- Audit Service
- Kill Switch Service
- Eval Harness
- Normalization Service · Validation Service
- Entity Resolution Service (framework; live with sources in Phase 1)
- Graph Writer · Structured Writer

Ingestion & enrichment (Phase 1):

- Source Connector Service (for CMDB, Vuln Scanner, PKI, Ownership)
- SBOM Adapter (for Black Duck output)

- Build Event Adapter (basic — image and artifact metadata only; SLSA attestation processing is Phase 3)
- Provenance Verifier (signature verification in advisory mode; enforcement at deploy gate is Phase 3)
- Enrichment Service (baseline; threat actor enrichment is Phase 2)
- Risk Scoring Service
- Supply Chain Risk Scorer
- Embedding Pipeline
- Indicator Matching Service (scaffolded: API surface and OpenSearch indexes, but no live indicators until Phase 2)

Query orchestration (Phase 1):

- Intent Router · Query Planner
- Graph Query Service · NL2SQL Service · SQL Gateway · Vector RAG Service
- Transitive Closure Service (bounded synchronous + top-1% materialization)
- Result Fusion Service (graph + SQL + vector; indicator-match fusion lights up in Phase 2)

Use case & agent runtime (Phase 1):

- Use Case APIs: Vulnerability Chain · PKI Risk · Supply Chain Risk
- Agent Supervisor
- Specialist Agents: Topology Analyst · Vulnerability Chain · PKI Intelligence · Evidence · Supply Chain Risk

Services deferred to Phase 2

- **TI Feed Adapter** (active ingestion from approved threat intel feeds)
- **Indicator Matching Service** transitions from scaffolded to live
- **TI Source Trust Registry** transitions from scaffolded to live trust scoring
- **Zero-Day Triage Scorer**
- **Use Case APIs:** Zero-Day Triage · Blast Radius · Exposure Analysis
- **Specialist Agents:** Zero-Day Triage · Ownership Resolution
- Additional Source Connectors: Endpoint (CrowdStrike/Tanium), DNS/IPAM, Load Balancers, Cloud Inventory (AWS/Azure/GCP), Kubernetes/OpenShift
- Enrichment Service additions: threat actor attribution, behavior-mapped TTP enrichment
- Result Fusion Service extension to include indicator-match outputs
- Kill Switch Service extension: per-TI-source kill switches

Services deferred to Phase 3

- **Provenance Verifier** transitions from advisory mode to enforcement at deploy gate
- **Build Event Adapter** extension: full SLSA / in-toto attestation processing
- **Transitive Closure Service** extension: full pre-materialization beyond top 1%
- Vendor / TPRM integration (Source Connector additions for third-party data)
- Behavior / TTP matching via SIEM bridge (Indicator Matching Service modality extension)

Success criteria — additions

Metric	Target
SBOM coverage of prod images	≥ 90%
Supply chain closure precision (sampled)	≥ 0.95
Supply chain ranking precision@10	≥ 0.85
Synchronous closure latency p95	≤ 30 s for top 1% packages

Metric	Target
Supply chain analyst productivity	“Which apps use package X” answered ≤ 30 s vs current days

Kill criteria — additions

- SBOM coverage of prod images remains below 70% at the end of Phase 1 month 2 (foundation issue with the build pipeline that the platform cannot route around).
- Bounded transitive closure latency stays above 2 min p95 after tuning (architectural issue with the graph store or query approach).

---

23. Phased roadmap

Re-balanced: Phase 1 includes supply chain (high value, leverages existing Black Duck investment); zero-day triage moves to Phase 2 after TI infrastructure.

Phase	Duration	Scope	Exit criteria
Phase 0 — Foundation	8 weeks	**Services stood up:** Event Bus, Ontology/Schema/Model/TI Source Trust Registries, Policy Service (PDP), Audit Service, Kill Switch Service, Eval Harness, Normalization Service, Validation Service, ER framework, Graph Writer, Structured Writer. **Scaffolded (interfaces + persistence only, not live):** Indicator Matching Service, TI Source Trust scoring. **Cross-cutting:** Ontology v1 incl. supply chain extensions, ER gold set, tenant model + adversarial suite, OpenShift baseline, Neo4j Enterprise + vector store + Postgres + BigQuery + OpenSearch deployed	Platform skeleton with sample data, adversarial suite passing, ER framework benchmarked, governance bodies stood up

Phase	Duration	Scope	Exit criteria
Phase 1 — MVP / Pilot	14 weeks	**Services going live:** Source Connector Service (CMDB, Vuln, PKI, Ownership), SBOM Adapter, Build Event Adapter (basic), Provenance Verifier (signature verify, advisory mode), ER live with sources, Enrichment Service (baseline), Risk Scoring Service, Supply Chain Risk Scorer, Embedding Pipeline, Transitive Closure Service (bounded sync + top-1% pre-mat), Intent Router, Query Planner, Graph Query Service, NL2SQL Service, SQL Gateway, Vector RAG Service, Result Fusion (graph+SQL+vector), Use Case APIs (Vuln Chain, PKI Risk, Supply Chain Risk), Agent Supervisor, 5 Specialist Agents (Topology, Vuln Chain, PKI, Evidence, Supply Chain Risk). **Cross-cutting:** governed NL2SQL, basic RAG, pilot UI incl. supply chain dashboard	Pilot teams validate; supply chain reach query operational; sign-off from cyber sponsor, MRM committee, security architecture

Phase	Duration	Scope	Exit criteria
Phase 2 — Threat Intel + Expand	12 weeks	**Services going live:** TI Feed Adapter, Indicator Matching Service (was scaffolded), TI Source Trust Registry (was scaffolded), Zero-Day Triage Scorer, Zero-Day Triage Agent, Ownership Resolution Agent, Use Case APIs (Zero-Day Triage, Blast Radius, Exposure Analysis). **Services extended:** Enrichment Service (+ threat actor attribution), Result Fusion (+ indicator-match), Kill Switch Service (+ per-TI-source). **Source coverage added:** Endpoint, DNS/IPAM, load balancers, cloud inventory, OpenShift/K8s	Zero-day triage operational with TI sponsor sign-off; source coverage ≥ 70% of priority list; new use cases meet quality targets

Phase	Duration	Scope	Exit criteria
Phase 3 — Hardening + Supply Chain Maturity	10 weeks	**Services extended:** Provenance Verifier (advisory mode → enforcement at deploy gate), Build Event Adapter (+ full SLSA/in-toto attestation processing), Transitive Closure Service (+ full pre-materialization beyond top 1%), Indicator Matching Service (+ behavior/TTP modality via SIEM bridge). **New integrations:** Vendor / TPRM platform via Source Connector additions. **Cross-cutting:** HA & DR drills, full SLO enforcement, ReBAC for ownership delegation, agent autonomy gates loosened with HITL fallback, performance tuning at scale	Production SLOs met for 30 days; DR drill passed; provenance enforcement live in shadow mode
Phase 4 — Steady State	Ongoing	Self-service source onboarding, capability marketplace, automated evidence pack generation, ontology v2 candidate, firmware supply chain exploration	Self-service onboarding median < 5 days; ≥ 3 third-party engines registered

Deprecated over time

- Manual spreadsheet correlation
- Static vulnerability and supply chain exports
- One-off certificate impact reports
- Uncontrolled analyst SQL queries
- Duplicative topology dashboards
- Per-tool authorization patchwork
- **War-room “do we have this?” exercises during zero-day events**
- **Manual SBOM grep at incident time**

24. Day 2 operations & runbooks

v2 §24 specified runbooks for ER quality regression, tenant misclassification (Sev-1, immediate kill switch + audit-trail review + CISO+compliance notification within 1h), connector data quality drop, ontology change rollout (with mandatory dual-write window), model regression with kill switch and canary rollback, and cross-tenant audit request fulfillment. The v3 runbooks below cover the new threat-intel and supply-chain scenarios.

Runbook: Zero-day declared

```
Trigger: High-severity advisory ingested from any approved feed, OR IR lead declares
         zero-day status, OR external advisory cross-references a known internal exposure
Severity: Sev-1 (advisory severity ≥ critical) | Sev-2 (high)
Owner: TI on-call + IR lead

Actions:
1. Advisory auto-ingested; Indicators published to graph + OpenSearch (T1).
2. Indicator Matching Service runs sweep across all modalities (auto).
3. Zero-Day Triage Agent produces initial ranked exposure list (< 15 min p95).
4. TI on-call validates source trust and indicator confidence; suppresses any obvious
   false matches; recalibrates if needed.
5. Owner notifications dispatched for top-N ranked findings via approved channels.
6. IR lead reviews aggregate exposure across tenants; declares incident if warranted.
7. Per-tenant detail access granted to affected tenant security leads under
   burst-mode authorization with linked incident ticket.
8. Triage findings drive standard remediation flow (tickets, SLAs).
9. After-action: indicator effectiveness reviewed; feed trust scores updated; gold set
   augmented with this advisory + observed matches.
```

Runbook: Supplier compromise advisory

```
Trigger: Compromise advisory for a package, vendor, or build pipeline component
Severity: Sev-1 (active exploitation reported) | Sev-2 (compromise without exploit) |
          Sev-3 (suspicious indicators only)
Owner: App security on-call + supply chain owner

Actions:
1. Advisory normalized; affected PackageVersion(s) identified by purl + version constraint.
2. Bounded transitive closure (depth ≤ 8) initiated synchronously for high-priority tenants.
3. If estimated closure is deep or wide, async deep closure dispatched with reserved capacity.
4. Supply Chain Risk Agent ranks affected applications.
5. App security lead reviews ranking; validates a 10% sample manually.
6. Build provenance for affected artifacts re-verified:
   - Were artifacts built before/after compromise window?
   - Are signatures still valid?
   - Was build pipeline itself compromised?
7. If build pipeline compromised: ALL artifacts produced in window are suspect;
   broaden scope and re-rank.
8. Owner notifications dispatched with upgrade-path remediation suggestions where
   available.
9. Affected packages quarantined in private registry (manual action; automated in Phase 3).
10. Track remediation through standard ticketing; SLA driven by Sev tier.
11. After-action: closure correctness reviewed; pre-materialization decision for this
    package re-evaluated; SCA tool feedback if SBOM gaps identified.
```

Runbook: TI feed quality regression

```
Trigger: TI Source Trust Registry detects FP rate spike or source-trust drop > 0.2
Severity: Sev-2
Owner: TI team

Actions:
1. Auto-suppress indicators from feed below confidence threshold.
2. Investigate: feed compromise? Schema change? Feed-side data quality issue?
3. Notify feed vendor if commercial.
4. If suspected compromise: kill switch on feed (manual reactivation).
5. Re-validate indicators from feed in current actionable window; relabel matches.
6. Recompute source trust; communicate to analysts.
7. Post-incident: feed contract review; consider replacement or de-prioritization.
```

Runbook: SBOM coverage drop

```
Trigger: SBOM coverage of prod images < 85% for 3 days
Severity: Sev-3 (escalate to Sev-2 if < 70%)
Owner: App security + platform engineering

Actions:
1. Identify images without SBOM via conformance dashboard.
2. Group by build pipeline; identify systemic vs one-off issues.
3. Engage build platform team for systemic gaps.
4. For one-off: generate SBOM out-of-band via Syft/Trivy on registry images.
5. Backfill SBOMs into platform.
6. Strengthen pre-deploy SBOM check (warn → block in Phase 3).
```

Runbook: Transitive closure pre-materialization drift

```
Trigger: Closure correctness eval shows materialized closure diverges from live by > 1%
Severity: Sev-2
Owner: Platform team

Actions:
1. Force live traversal for affected package family (bypass materialization).
2. Investigate incremental refresh logic (which events should have triggered refresh?).
3. Rebuild materialization for affected family.
4. Add regression test for the missed event type.
```

---

25. Ontology & governance processes

v2 §25 established ontology governance: a single Ontology Owner on the Cyber Platform team, a Change Advisory Board (cyber architecture lead, data governance lead, security architecture lead, rotating use-case engine owner, eval harness owner) chaired by the Ontology Owner, SemVer (PATCH for new optional properties, MINOR for new node/relationship types or required properties with default, MAJOR for removals/renames/cardinality changes), MAJOR changes require ≥ 7-day dual-write + dual-read window and ≥ 30-day deprecation window, ontology version stamped on every node/edge, quarterly review of unused types for deprecation, and parallel governance models for the Schema Registry (sources) and Model Registry (LLMs, embeddings, scorers).

In v3, the CAB now includes a TI/IR representative and a software supply chain representative when changes touch threat or supply chain ontology subspaces.

Subspace ownership

Ontology subspace	Owner
Assets, applications, services	Cyber Platform team
Vulnerabilities, exposures	Vuln Mgmt + Cyber Platform
Identity, credentials	Identity team
Data classification	Data Governance
Threat, indicators, advisories	TI team
Packages, SBOM, build provenance	App Security + Build Platform
Third party, vendor	App Security + Vendor Mgmt

A single Ontology Owner coordinates across subspaces and chairs the CAB.

---

26. Alternatives considered

v2 §26 evaluated and rejected the following primary alternatives: relational-only (multi-hop traversal is painful, attack-path analysis is unnatural — relational is kept for metadata and analytics, not primary); Neo4j-only (weak aggregation/reporting, no good NL2SQL surface, awkward vector retrieval); lake-first with graph view (real-time topology weak); commercial exposure platforms like Wiz / Axonius / Panaseer (rejected as replacement due to data residency, on-prem constraints, customization limits, vendor lock-in on the most strategic asset; kept as benchmark and potential ingestion source for cloud-only scopes); SOAR / SIEM extension (SOAR is procedural, SIEM is log-centric — both lack the relationship-first model); outsourced ontology + ER to vendor (loses control of the most-leveraged platform asset, vendor model risk, data leaves perimeter). The chosen approach is polyglot persistence with Neo4j for topology, BigQuery + Postgres for structured analytics and metadata, an approved vector store for semantic search, OpenSearch for text/log search, an object store for evidence and replay, Kafka for ingestion, a supervisor + specialist agent runtime over typed APIs, composite tenancy, and governance-first ontology / ER / model risk.

The v3 additions below evaluate alternatives specific to zero-day and supply chain.

Zero-day + supply chain specific

Alternative	Pros	Cons	Decision
Rely on SIEM correlation rules for zero-day	Existing tool, no new infra	No topology context, no ownership routing, no transitive analysis	Rejected as primary; integrate as a signal source
Use SCA tool’s own dashboard for supply chain	No new build effort	Limited topology context, no cross-tool correlation, vendor lock-in on the most strategic graph	Rejected as primary; SCA tools remain SBOM source

Alternative	Pros	Cons	Decision
Buy a commercial SBOM platform (Anchore, Chainguard, Endor Labs)	Mature features, faster time-to-value	Vendor-lock argument as v2; weak integration with topology graph; data residency	Considered as ingestion source; not as orchestration layer
Build closure with relational recursive CTEs	Mature SQL infra	Performance and maintainability at depth 12 are poor; no graph algorithms	Rejected; Neo4j is the right tool
Outsource zero-day triage to MDR vendor	Reduces internal load	Vendor latency, no organizational context, expensive at scale, leaks topology to vendor	Rejected for primary path; MDR remains for off-hours augmentation

---

27. Open questions & risks

v2 §27 raised twelve open questions, summarized here for context: (1) which CMDB fields are trusted as authoritative ownership, (2) which vuln scanner is source of truth where they conflict, (3) which PKI system is authoritative, (4) which approved vector store on OpenShift, (5) which BigQuery datasets are in scope for initial NL2SQL, (6) which classifications require masking and at which layer, (7) confidence threshold for analyst-facing outputs, (8) cross-tenant approver rotation and SLA, (9) MRM committee cadence and SLAs, (10) source contract conformance enforcement teeth (block vs warn), (11) OpenShift node sizing and capacity reservation for Neo4j Enterprise, (12) whether Cypher generator will be in scope at any phase as a Tier 1 model.

v3 adds:

#	Question / Risk	Owner	Decision needed by	Impact if unresolved
13	Which TI feeds are approved for production ingestion?	TI team + Risk	Phase 1 wk 6	Phase 2 timing
14	Internal TI aggregator (MISP, OpenCTI, custom)?	TI team	Phase 1 wk 8	Architecture decision for Phase 2
15	SBOM authoritative source where Black Duck and Snyk conflict?	App Security	Phase 0 wk 6	Conflicting closure results
16	Build provenance enforcement timeline: shadow mode → block?	Build Platform + App Security	Phase 2	Risk vs developer friction

#	Question / Risk	Owner	Decision needed by	Impact if unresolved
17	Vendor/third-party data sourced from TPRM platform — which one?	Vendor Mgmt	Phase 2 wk 4	Vendor risk coverage delay
18	Transitive closure depth cap: hard limit or analyst-overridable with approval?	Platform leadership + App Security	Phase 1 wk 4	Operational flexibility vs DoS risk
19	False-positive feedback loop: how does suppression label affect source trust?	TI team + ML team	Phase 2 wk 4	TI quality
20	Cross-tenant burst mode rate limits and approval matrix?	Compliance + TI	Phase 1 wk 8	Phase 2 enablement
21	Behavior/TTP matching depth: full MITRE ATT&CK or curated subset?	TI team + SIEM team	Phase 2 wk 2	Scope and SIEM integration cost
22	SBOM generation gap for legacy / vendored binaries — manual inventory?	App Security	Phase 1 wk 8	Coverage shortfall

---

28. Appendix

Glossary

Term	Meaning
ABAC	Attribute-Based Access Control
Advisory	A formal security notification (vendor PSIRT, CISA KEV, internal IR)
AST	Abstract Syntax Tree
Attestation	Signed claim about an artifact (in-toto, SLSA)
CAB	Change Advisory Board
CMDB	Configuration Management Database

Term	Meaning
CPE	Common Platform Enumeration
CVE	Common Vulnerabilities and Exposures
Cypher	Neo4j graph query language
ER	Entity Resolution
HITL	Human-in-the-Loop
IOC / Indicator	Indicator of Compromise — observable signal of malicious activity
KEV	CISA Known Exploited Vulnerabilities catalog
MISP / OpenCTI	Open-source threat intelligence platforms
MITRE ATT&CK	Standard taxonomy of adversary tactics, techniques, procedures
MRM	Model Risk Management
OBO	On Behalf Of (token exchange pattern)
OPA	Open Policy Agent
PDP	Policy Decision Point
PKI	Public Key Infrastructure
PSIRT	Product Security Incident Response Team (vendor)
PURL	Package URL — canonical cross-ecosystem package identifier
RAG	Retrieval-Augmented Generation
ReBAC	Relationship-Based Access Control
RFC 8693	OAuth 2.0 Token Exchange
SBOM	Software Bill of Materials
SCA	Software Composition Analysis
SLO	Service Level Objective
SLSA	Supply-chain Levels for Software Artifacts (build provenance framework)
SR 11-7	Federal Reserve / OCC guidance on model risk management
SPIFFE	Secure Production Identity Framework For Everyone
STRIDE	Threat model categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TI	Threat Intelligence
TPRM	Third-Party Risk Management
TTP	Tactics, Techniques, and Procedures (MITRE ATT&CK)

Reference architecture mantra (extended)

- **Neo4j** answers: *how are things connected — including transitive dependencies?*
- **NL2SQL / BigQuery** answers: *how many, which rows, what status, what trend?*
- **Vector store** answers: *what does this mean, where is it documented?*
- **OpenSearch + Indicator Index** answer: *where does this string / hash / IP appear?*
- **Transitive Closure Service** answers: *what's the full reach of this package, transitively?*
- **Indicator Matching Service** answers: *does anything we own match this signal?*

- **OpenShift** provides: *private-cloud runtime and operational controls.*
- **Kafka** provides: *real-time, replayable discovery and topology updates.*
- **Policy service** provides: *tenant, ownership, classification, and burst-mode enforcement.*
- **TI Source Trust Registry** provides: *the difference between actionable signal and noise.*
- **Ontology + ER + governance** provide: *the difference between a trusted topology and an interesting demo.*

— End of document.

---

29. Reference: agents, tenant APIs, use case registration

This section consolidates three operational references that are referenced across the document. They are reference material, not new design — but having them in one place simplifies design review and onboarding.

29.1 Agents inventory

The platform has one supervisor and up to nine specialist agents across MVP and Phase 2. Each row references the SR 11-7 tier from §20 and the capability bounds from §15.

Agent	Phase	Specialty	Tools allowed	SR 11-7 tier	HITL required for
Agent Supervisor	MVP (P1)	Plan, delegate, synthesize across specialists	Specialist delegation API, Intent Router, Policy PDP	2	Cross-tenant queries (excl. pre-authorized burst), high-cost queries, ambiguous intent
Topology Analyst	MVP (P1)	Free-form topology questions and exploration	Graph Query, Vector RAG	2	none in MVP
Vulnerability Chain	MVP (P1)	Vuln chain ranking and remediation prioritization	Graph Query, NL2SQL, Risk Scoring, Vector RAG	1	When ranking influences remediation SLA
PKI Intelligence	MVP (P1)	Cert expiry, weak crypto, downstream impact	Graph Query, NL2SQL	2	Proposing revocation candidates
Evidence	MVP (P1)	Pull evidence, runbooks, schema docs for analyst	Vector RAG, OpenSearch, Object Store read	3	none

Agent	Phase	Specialty	Tools allowed	SR 11-7 tier	HITL required for
Supply Chain Risk	MVP (P1)	Compromise advisory triage with transitive closure	Transitive Closure, Graph Query, NL2SQL, Provenance Verifier, SC Risk Scorer	**1**	Proposing image rebuild orders, vendor compromise flagging, closures beyond depth 12
Zero-Day Triage	Phase 2	Advisory → environment exposure with fuzzy match	Indicator Match, Graph Query, Zero-Day Scorer, Vector RAG, TI Source Trust Registry	**1**	Declaring affected when source trust < 0.7, cross-tenant escalation, indicator suppression
Ownership Resolution	Phase 2	Resolve and route ownership for orphaned findings	Graph Query, SQL Gateway	2	Proposing new owners

Specialists never call each other directly. All inter-agent flow goes through the Supervisor via typed delegation contracts (§15).

29.2 Tenant-facing APIs

APIs exposed to tenant users (analysts, app owners, PKI team, etc.) and to tenant-scoped service accounts. All require OIDC authentication, all require tenant scope, all are read-only against stores in MVP and Phase 1.

API	Method + path	Use case	Phase	Authorized roles
Hybrid query	`POST /v1/query/hybrid`	Natural language question routed to graph + SQL + vector	MVP (P1)	All tenant analyst roles
Graph query	`POST /v1/query/graph`	Template-driven graph traversal	MVP (P1)	All tenant analyst roles
SQL execute	`POST /v1/query/sql/execute`	Governed read-only SQL (AST-validated)	MVP (P1)	All tenant analyst roles
Vector retrieval	`POST /v1/query/vector`	Tenant-scoped semantic retrieval	MVP (P1)	All tenant analyst roles
Vulnerability chain	`POST /v1/vuln-chain/analyze`	Ranked vuln paths with topological scoring	MVP (P1)	`security_analyst`, `vuln_mgmt`, `app_owner`

API	Method + path	Use case	Phase	Authorized roles
PKI risk	`GET /v1/pki/risk`	Expiring/weak cert impact analysis	MVP (P1)	`pki_team`, `app_owner`, `security_analyst`
Supply chain impact	`POST /v1/supply-chain/impact`	Transitive closure on a package version	MVP (P1)	`software_supply_chain_owner`, `app_owner`, `vuln_mgmt`
Zero-day triage	`POST /v1/zero-day/triage`	Advisory → ranked exposure list	Phase 2	`threat_intel_analyst`, `incident_response_lead`
Blast radius	`POST /v1/blast-radius/analyze`	Reachability from a compromised asset	Phase 2	`security_analyst`, `incident_response_lead`
Exposure analysis	`POST /v1/exposure/analyze`	Internet/identity/data exposure of an app	Phase 2	`app_owner`, `security_analyst`
Evidence	`GET /v1/evidence/{lineage_id}`	Fetch evidence pack for a result	MVP (P1)	All tenant analyst roles (within their tenant scope)
Cross-tenant aggregate	`POST /v1/cross-tenant/aggregate`	Pre-approved aggregate templates across tenants	MVP (P1)	`cyber_leadership`, `auditor`, `cyber_global_analyst`

Admin and platform APIs (registry edits, kill switch toggles, eval harness runs, agent capability matrix changes, ontology/schema/model registry CRUD) are explicitly **not** tenant-facing. They are control plane operations gated by separate roles (`platform_admin`, `cyber_platform_engineer`, `ontology_owner`, `mrm_committee`) and go through CAB review (§25).

29.3 Use case registration

A use case is the unit of capability extension. The §22 MVP and Phase 2 use cases were authored against this registration model; future use cases follow the same workflow.

Use Case Registry record

Every use case is registered as a versioned record in the Use Case Registry (a control-plane registry alongside Ontology, Schema, Model, and TI Source Trust). Schema:

```yaml
use_case_id: vuln_chain
version: 1.2.0
status: live                    # proposed | implementing | canary | live | deprecating | retired
owner_team: cyber-platform
business_sponsor: cyber-sponsor@bank
mrm_partner: mrm-committee@bank

scope:
  target_tenants: [all]         # or specific list
  target_environments: [prod, nonprod]
  target_classifications: [public, internal, confidential, restricted]
  cross_tenant: false           # set true only for explicitly cross-tenant use cases

contract:
```

```yaml
  api_endpoint: POST /v1/vuln-chain/analyze
  typed_inputs_schema: schemas/vuln_chain_inputs.proto
  typed_outputs_schema: schemas/vuln_chain_outputs.proto
  evidence_required: true

components:
  use_case_engine: vuln-chain-engine@1.2.0
  agents: [vuln-chain-agent@1.4.0]
  tools_required: [graph_query, nl2sql, sql_gateway, risk_scoring, vector_rag]
  ontology_dependencies:
    min_version: 1.0.0
    node_types: [Asset, Finding, CVE, Service, BusinessApplication, NetworkExposure, DataClassification]
    relationship_types: [HAS_VULNERABILITY, MANIFESTATION_OF, RUNS_ON, CONNECTS_TO, EXPOSED_TO]
  source_dependencies:
    required: [cmdb, vuln_scanner, ownership]
    optional: [endpoint_security, network_inventory]

governance:
  sr_11_7_tier: 1
  hitl_gates:
    - when: "ranking influences remediation SLA"
    - when: "result confidence < 0.7 on top-3 findings"
  policy_requirements:
    rbac_roles: [security_analyst, vuln_mgmt, app_owner]
    abac_attributes: [authorized_tenants, authorized_classifications]

evals:
  suite: vuln-eval-v1
  gold_set_size: 200
  gating_metrics:
    - {name: precision_at_10, threshold: 0.85, type: gate}
    - {name: tenant_boundary_pass, threshold: 1.0, type: gate}
    - {name: ranking_correlation_with_sme, threshold: 0.70, type: alert}
  drift_detection:
    cadence: weekly
    alert_threshold: 2pp_drop
    page_threshold: 5pp_drop

operations:
  budget:
    max_cost_units_per_call: 2500
    max_p95_latency_ms: 8000
  kill_switch_id: vuln_chain
  monitoring_dashboard: https://...
  on_call_rotation: cyber-platform-oncall

audit:
  classification: internal
  retention_years: 7

lifecycle:
  registered_at: 2026-04-15
  last_cab_review: 2026-04-22
  next_periodic_review: 2027-04-22    # Tier 1 → annual minimum
  deprecation_announced_at: null
  retired_at: null
```

Registration workflow

The four-phase workflow shown in the diagram above breaks down as:

Phase 1 — Definition (owner: product + cyber sponsor)

The proposal is a short document that answers four questions: what problem does this solve, for whom, what answer does it produce, and what would be true if we didn't build it. The typed spec defines the API contract — inputs, outputs, evidence requirements, target tenants, target classifications, cross-tenant scope. The gap analysis identifies what the platform is missing to

support the use case: missing ontology elements, missing tools, missing data sources, missing agents. Gaps become tracked work items that must close before Phase 2 begins. Gate to exit Phase 1: spec signed off by sponsor and architecture review, all gaps either closed or scheduled.

Phase 2 — Implementation (owner: platform engineering + MRM partner)

Three parallel workstreams: (a) implement the use case engine — the API endpoint and domain logic that fronts the use case; (b) implement or reuse a specialist agent with bounded tool access and budget caps; (c) implement the eval suite — the gold set, the metrics, the gating thresholds, and the adversarial suite. In parallel, the MRM partner assigns the SR 11-7 tier (Tier 1 if the output directly drives remediation prioritization or incident response, Tier 2 for analyst-facing decisions, Tier 3 for purely informational or monitoring outputs). HITL gates are defined for low-confidence outputs, cross-tenant escalation, and any output that proposes a write action. Gate to exit Phase 2: eval suite passes thresholds, tenant-boundary adversarial suite is 100%, MRM tier validated, security architecture review complete.

Phase 3 — Onboarding (owner: CAB chair + Ontology Owner)

The use case record is created in the Use Case Registry. CAB reviews the record: cyber architecture lead, data governance lead, security architecture lead, the eval harness owner, and the relevant use-case domain rep (e.g., TI rep for Zero-Day, supply chain rep for Supply Chain Risk). For Tier 1 use cases, MRM committee sign-off is mandatory before canary; Tier 2/3 require tech lead sign-off. Canary rollout follows the standard 1% → 10% → 100% pattern over 5 days, with eval harness and tenant-boundary tests running continuously. Any regression aborts the canary and the use case returns to Phase 2. Gate to exit Phase 3: canary metrics clean for full duration, no eval regression, no audit anomalies.

Phase 4 — Live (owner: use case engine owner + SRE on-call)

The use case is live. Eval drift detector runs at the suite's defined cadence (weekly default; daily for Tier 1). Kill switch is armed and tested quarterly. Cost and latency SLOs are tracked on dashboards. Tier 1 use cases get quarterly periodic re-review by the CAB; Tier 2/3 annual. Any new ontology version, new model version, or new tool added to the agent triggers a re-evaluation against the registered gating thresholds. Failed re-evaluation triggers either patching or controlled rollback.

Deprecation

Same workflow in reverse. Deprecation is announced via the registry (status → `deprecating`, `deprecation_announced_at` timestamped), with a minimum deprecation window of 90 days during which the use case still serves but emits deprecation warnings in responses. At end of window, kill switch is engaged, traffic drains, and the registry record transitions to `retired`. Audit and evidence retention continue per the use case's audit classification.

Why this matters

Without registration, a use case is undocumented platform tribal knowledge — and that's how an analyst-facing tool ends up being load-bearing for an incident response decision with no one having validated that it's appropriate for that purpose. The registry is what lets the platform answer "is this use case Tier 1 or Tier 3?", "what evals gate it?", "who owns it when it regresses at 2am?" — and it's what lets MRM, audit, and regulators see the full surface of decision-support that the platform exposes.

29.4 NL2SQL access patterns

NL2SQL is one of the most heavily-governed paths in the platform (§16) and one of the most frequently invoked. It is **not** exposed as a directly callable endpoint to tenant users. Four distinct access paths reach it, each with different governance posture and authorized callers.

Path 1: Hybrid Query API (dominant traffic)

The most common access pattern. An analyst submits a natural language question to the Hybrid Query API (`POST /v1/query/hybrid`); the Query Planner classifies the intent, and one of the subqueries it dispatches in parallel is an NL2SQL call to fetch structured data alongside the graph and vector subqueries. The analyst never directly invokes NL2SQL; they get a merged response. This is the parallel-fan-out path detailed in the hybrid query lifecycle (§11).

```
POST /v1/query/hybrid
Authorization: Bearer <user_token>
X-Correlation-ID: ...

{
  "question": "Open critical vulns on prod apps in Consumer Banking",
  "filters": {"environment": "prod", "severity": "critical"},
  "tenant_context": {"requested_tenant": "consumer-banking"}
}
```

Authorized roles: all tenant analyst roles (`security_analyst`, `vuln_mgmt`, `pki_team`, `app_owner`, `software_supply_chain_owner`).

Path 2: Use Case APIs (internal invocation)

When a Use Case API runs (Vuln Chain, PKI Risk, Supply Chain Risk in MVP; Zero-Day, Blast Radius, Exposure in Phase 2), its specialist agent needs structured data alongside graph results. The agent calls NL2SQL through its bounded tool API per the §15 capability matrix. The use case API is the visible surface; NL2SQL fulfills the structured-data slice of the request. Callers never see the NL2SQL hop.

Authorized roles: same as the use case API's authorized roles (see §29.2).

Path 3: Agent Supervisor delegation

When a user's natural language question routes through agent delegation (rather than direct hybrid query), the Supervisor dispatches to a specialist agent via the typed delegation contract (§15). The specialist may call NL2SQL as one of its tools. Same end result as Path 2 but reached through `supervisor → specialist → tool` instead of directly from a use case API.

Authorized roles: implicit — whatever the user's role authorized for the originating request, propagated via OAuth Token Exchange (§14).

Path 4: Direct SQL APIs (power user and programmatic)

For power users wanting to inspect generated SQL before execution, and for programmatic callers (SOAR playbooks, eval harness, dashboard backends) needing pinned queries:

Endpoint	Purpose	Authorized roles
`POST /v1/query/sql/generate`	Generate SQL from NL question without execution. Returns SQL + AST validation result + cost estimate + confidence	`cyber_platform_engineer`, named power users; service accounts via approved pattern
`POST /v1/query/sql/execute`	Execute a SQL statement (generated or pinned template). Still subject to AST validator, predicate injection, cost gating, and row caps	Same as above

Even direct execution does not bypass governance. The SQL Gateway is the choke point regardless of how the SQL string arrived: every SQL statement goes through AST parse, read-only check, mandatory tenant predicate injection, banned-construct rejection, column allowlist enforcement, row LIMIT injection, cost estimator, and final masking on results. The only thing Path 4 skips compared to Path 1 is the LLM generation step.

Example:

```
POST /v1/query/sql/generate
Authorization: Bearer <user_token>

{
  "question": "How many open critical findings per app owner in Consumer Banking?",
  "scope": {"tenants": ["consumer-banking"]},
  "preview_only": true
}
```

Response surfaces the generated SQL, the AST validation result, the cost estimate, and the confidence score, allowing the caller to review before separately calling `/v1/query/sql/execute`.

What is deliberately not exposed

- **No direct LLM endpoint.** Callers cannot prompt the underlying NL2SQL model. The prompt is constructed by the NL2SQL Service from the question, the retrieved schema, and the user's scope — never from raw user input passed through.
- **No raw SQL execution mode that bypasses the SQL Gateway.** Even `/v1/query/sql/execute` runs the full AST validator. There is no path around the gateway.
- **No cross-tenant NL2SQL outside the approved template library.** Cross-tenant queries route through the §21 governance regardless of how they were generated. Pre-approved cross-tenant SQL templates can be invoked by `cyber_leadership`, `auditor`, and `cyber_global_analyst` roles; anything not in the approved library routes to HITL.
- **No NL2SQL access from outside the OpenShift estate** without going through the API gateway. Internal services use mTLS + workload identity; external clients hit the gateway with OIDC tokens.

Summary table

Path	Visible endpoint	Caller type	Generation?	Execution?	Governance
Hybrid Query	`/v1/query/hybrid`	Analyst portal, IR console	yes	yes	Full §16 path
Use Case API	`/v1/vuln-chain/analyze`, `/v1/pki/risk`, etc.	Agent calls NL2SQL internally	yes	yes	Full §16 path
Agent Supervisor	(via supervisor → specialist)	Triggered by user query routed to delegation	yes	yes	Full §16 path
Direct generate	`/v1/query/sql/generate`	Power user, programmatic	yes	no	Validator + cost estimator run; no execution

Path	Visible endpoint	Caller type	Generation?	Execution?	Governance
Direct execute	`/v1/query/sql/execute`	Power user, programmatic, SOAR	no	yes	Full §16 path; LLM bypassed but gateway not

— End of section 29.

---

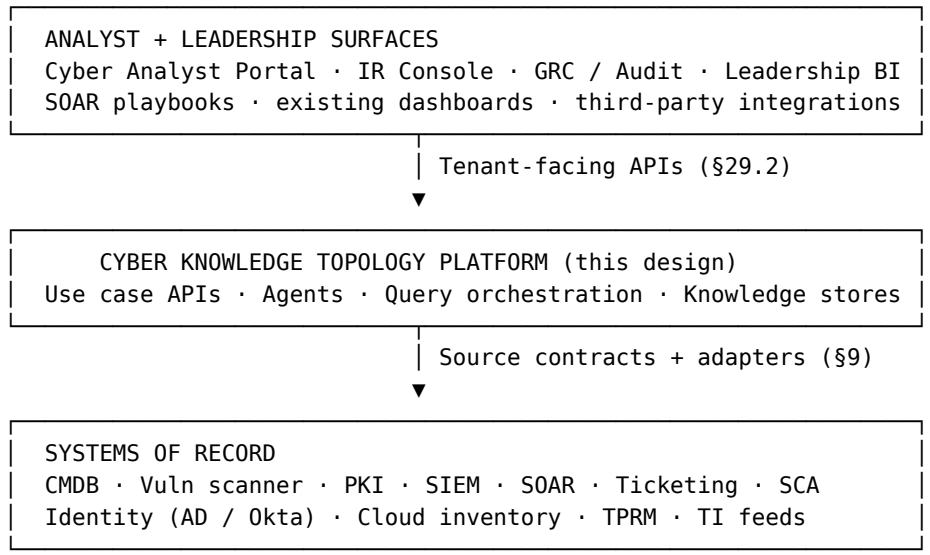
30. Integration with the enterprise cyber platform

The platform does not live on an island. It sits within an existing enterprise cybersecurity stack and integrates with the systems already in production. This section makes explicit where it fits, what it integrates with, and how users and machine consumers reach it.

Position in the stack

The platform is a **horizontal capability layer**, not a vertical tool. It does not replace any existing system; it links them and exposes a unified topology + query surface above them. The right mental model is:

- **Existing vertical tools** (CMDB, vuln scanner, PKI, SIEM, SOAR, ticketing, SCA, TPRM, identity provider) remain authoritative systems of record. They keep doing what they do.
- **This platform** sits underneath the analyst experience and above the systems of record. It reads from those systems, links them via the ontology (§6), and exposes governed APIs and use case engines.
- **Consumers above this platform** include the cyber analyst portal, IR console, SOAR playbooks, leadership BI, GRC/audit tooling, and dedicated platform UIs (Supply Chain Dashboard, Zero-Day Triage Console).



The platform owns one capability — topology-aware querying with governance — and exposes it as APIs. Existing surfaces keep their UIs; nothing new replaces existing analyst consoles unless those teams choose to migrate workflows over time.

Four integration boundaries

Integration with the enterprise cyber platform happens in four directions, each with its own contract and lifecycle.

1. Read from systems of record (inbound data ingestion)

The data ingestion pipeline (§10) pulls or accepts pushed events from systems of record. Each source has a signed data contract (§9), an adapter, and a freshness SLA. The platform is a *consumer*, not a peer:

- CMDB stays authoritative for asset ownership; the platform ingests it.
- Vulnerability scanner stays authoritative for findings; the platform ingests them.
- PKI stays authoritative for certificate inventory; the platform ingests it.
- SBOM scanner (Black Duck) stays authoritative for component data; the platform ingests CycloneDX/SPDX output.
- The platform never writes back to systems of record in MVP or Phase 1.

2. Expose APIs upward to consumer surfaces

The 12 tenant-facing APIs in §29.2 are how everything above the platform talks to it:

- **Cyber analyst portal** calls `/v1/query/hybrid` for free-form questions and topology exploration.
- **SOAR playbooks** call `/v1/vuln-chain/analyze`, `/v1/supply-chain/impact`, and (Phase 2) `/v1/zero-day/triage` as part of their automated workflows.
- **Leadership BI dashboards** call `/v1/cross-tenant/aggregate` for posture rollups.
- **PKI team dashboards** call `/v1/pki/risk` for certificate impact analysis.
- **Audit and GRC tools** call `/v1/evidence/{lineage_id}` to fetch evidence packs.
- **Zero-Day Triage Console** (Phase 2, platform-operated UI) calls `/v1/zero-day/triage` and burst-mode cross-tenant endpoints.

Every API is OIDC-authenticated, scope-resolved, audited.

3. Trigger events downstream into other cyber systems

When the platform produces a ranked finding, an impact subgraph, or a remediation recommendation, it does not *act* on it — it emits events that existing systems act on:

- High-confidence vuln chain rankings push prioritized records into the ticketing system via outbound webhook.
- Zero-day triage results fire SOAR playbook triggers via an event queue.
- PKI risk findings create Venafi tickets through the existing PKI ticketing integration.
- Supply chain compromise results push image-rebuild requests to the build platform via webhook.

These integrations use **outbound webhooks or message queues**, never direct database writes against downstream systems. The platform recommends; downstream systems decide and act. This boundary is what keeps the platform's read-only posture clean while still being operationally useful.

4. Federate with identity and policy

The platform uses enterprise SSO (OIDC) for human authentication via the existing IdP. Workload identity uses SPIFFE within the OpenShift estate, matching the bank's existing pattern. The platform does not maintain its own identity store; it consumes group membership and role assignments from the enterprise directory. Policy hooks call out to the enterprise PDP where appro-

priate; the platform-internal Policy Decision Point handles fine-grained tenant + classification + tool-level decisions but defers to enterprise policy for coarser questions.

How users access the platform

Three primary access modes, each with a different governance posture.

Mode 1: Through existing analyst surfaces (the dominant mode)

The cyber analyst portal, IR console, vulnerability management dashboard, PKI dashboard, leadership BI, and (Phase 2) Zero-Day Triage Console all call platform APIs in the background. The analyst experiences a familiar UI; the platform is invisible. This is the right pattern for most analysts because they don't need to learn a new tool — their existing workflow becomes topology-aware. Adoption is via incremental integration: an existing dashboard adds a new widget that calls a use case API, or an existing portal page adds a "show topology" action.

Mode 2: Through dedicated platform UIs (for specific use cases)

Two surfaces are operated by the platform team because their workflows don't have a natural existing home:

- **Supply Chain Dashboard** (MVP / Phase 1) — SBOM coverage view, transitive dependency reach, supplier compromise impact. Used by the software supply chain owner and app security team.
- **Zero-Day Triage Console** (Phase 2) — advisory ingestion, indicator match results, ranked exposure list. Used by the TI analyst and IR lead during advisory response.

These are first-party UIs built by the platform team and live alongside (not inside) other cyber consoles.

Mode 3: Through direct API calls (power users and machine consumers)

SOAR playbooks, scripted analysis, the eval harness, and named power users (`cyber_platform_engineer` role) call platform APIs directly via HTTP. This mode requires:

- OIDC authentication for human users; mTLS + workload identity for service accounts.
- All calls go through the API gateway — no direct service-to-service bypass from outside the OpenShift estate.
- Power user roles see direct `/v1/query/sql/generate` and `/v1/query/sql/execute` endpoints (§29.4 Path 4) which standard analyst roles do not.

What goes where: integration responsibilities

Concern	Owned by platform	Owned by enterprise cyber stack
Asset inventory authoritative source	No	CMDB
Vulnerability detection	No	Vuln scanner
Certificate inventory authoritative source	No	PKI system
Identity directory	No	Enterprise IdP / AD
SIEM detection rules	No	SIEM team
Ticketing workflow	No	ServiceNow / Jira
SBOM generation	No	SCA tool (Black Duck)

Concern	Owned by platform	Owned by enterprise cyber stack
Threat intelligence production	No	TI team + commercial feeds
Topology and relationship modeling	**Yes**	—
Ontology governance	**Yes**	—
Entity resolution across sources	**Yes**	—
Governed NL query (NL2SQL, NL → graph)	**Yes**	—
Risk scoring composition (vuln chain, PKI, supply chain)	**Yes**	—
Cross-source impact analysis	**Yes**	—
Topology audit / evidence trail	**Yes**	—

Onboarding path for a new consumer

When a new consumer (a portal team, a SOAR engineer, an analytics group) wants to use the platform:

1. **Discovery** — review tenant-facing API catalog (§29.2) and use case capabilities.
2. **Scope agreement** — define which tenants, classifications, and roles will be involved; gives the platform team the data needed for policy and rate-limit configuration.
3. **Service account or role provisioning** — for machine consumers, register a service account with mTLS identity; for human consumers, ensure their existing role maps to an authorized role in the catalog.
4. **Integration build** — consumer team builds against the documented APIs. Platform team provides API contracts, example requests, and a sandbox tenant for testing.
5. **Eval against gold queries** — for SOAR or automated consumers, the consumer's specific queries are added to the eval harness so regressions are caught before they affect that consumer.
6. **Production rollout** — canary at low rate, monitor, scale up.
7. **Operational handoff** — consumer team owns their integration; platform team owns the platform.

---

31. Network flow: data plane and control plane separation

The platform's runtime topology separates synchronous request handling (data plane) from configuration and governance (control plane). This separation is operationally important and worth making explicit in this design.

What runs on the data plane

The data plane is where every user query and every ingestion event flows. Components on the data plane are sized for latency, scaled horizontally, and subject to SLOs (§17).

Query path (top-down): API Gateway → Query Planner → Executors (Graph Query, NL2SQL + SQL Gateway, Vector RAG, Closure Service, Indicator Match in Phase 2) → Result Fusion → response.

Ingestion path (bottom-up): Source Connectors / Adapters (Source, SBOM, Build, TI Feed in Phase 2) → Kafka → Normalize → Validate → Entity Resolution (or Indicator Match for IOCs) → Enrich → Risk Scoring (Risk, SC Risk Scorer, Zero-Day Scorer in Phase 2) → Provenance Verifier → Graph Writer / Structured Writer → Stores.

Both paths terminate at the Knowledge Stores (Neo4j, Postgres, BigQuery, Vector DB, Object Store, Redis, OpenSearch in Phase 2). Queries read from stores; ingestion writes to them.

What runs on the control plane

The control plane holds the platform's configuration, identity, registries, governance, and observability. Components on the control plane change on a slower governance cadence and are reached by data plane services through read APIs (with short-TTL caching) for every request.

Control plane service	Purpose
Identity Provider (OIDC)	User authentication; integrated with enterprise SSO
Policy Decision Point	RBAC + ABAC + ReBAC evaluation; tool-level authorization
Ontology Registry	Versioned ontology catalog with CAB-governed changes
Schema Registry	Versioned source schemas with owner sign-off
Model Registry	LLMs, embeddings, scorers; version, owner, tier, status
TI Source Trust Registry (Phase 2 live)	Per-feed trust score, freshness, FP rate
Use Case Registry	Use case definitions with phase, owner, eval suite (§29.3)
Eval Harness	Gold-set evals, drift detection, regression alerts
Kill Switch Service	Per-feature, per-agent, per-model, per-TI-source kill switches
Audit Service	Hash-chained immutable audit log; 7-year retention

The load-bearing invariant

The control plane is on the request path for *reads*, never for *writes*. Every data plane service reads from the registries, the policy service, and the kill switch state on every request (with caching to bound the overhead). But *changes* to any of those — ontology version bumps, model promotions, policy edits, kill switch flips — go through CAB review and registry workflows on a slower governance cadence. They do not ride the request path.

This is what makes the platform safe to operate at scale:

- A bad policy edit cannot take down ingestion. The edit goes through CAB review; the deploy is canaried; the data plane reads the new version after it stabilizes.
- A model regression rollback does not require a graph restore. The Model Registry pins the prior version; data plane services pick up the change on next cache refresh.
- An ontology change does not break running queries. MAJOR changes get a dual-write + dual-read window (§25); data plane services read the version they were deployed against.
- A kill switch flip propagates within seconds via the cache TTL; the change itself was a deliberate operator action.

Special cases

A few interactions don't fit cleanly into either plane:

- **Audit is bidirectional.** Every data plane region *writes* to Audit on every action. Audit is conceptually control plane (it's governance, not query traffic) but it is hot path for writes. That asymmetry is fine because audit writes are append-only and the audit store is sized accordingly.
- **Eval Harness samples from Stores independently.** It does not intercept queries; it reads from Stores directly to compute drift metrics. This means eval load is bounded and predictable, and a slow eval run cannot affect query latency.
- **Kill switch reads are the most latency-sensitive.** Every request checks active kill switches. The Kill Switch Service has the shortest cache TTL (a few seconds) so a flip propagates quickly. This is the one control plane component where stale reads have real consequences.

Why this matters for operations

The data plane / control plane separation maps directly to operational ownership and on-call rotations:

Plane	Owner	On-call
Data plane (query path)	Platform engineering	Cyber Platform SRE — query latency, error rate, freshness SLOs
Data plane (ingestion path)	Platform engineering	Cyber Platform SRE — ingestion lag, contract conformance
Control plane registries	Domain owners (ontology owner, model registry owner, etc.)	Domain owner during business hours; platform SRE for outages
Control plane governance	CAB chair + MRM partner	Not on-call in the traditional sense; CAB convenes on schedule
Audit	Compliance + platform SRE	Platform SRE for outages; compliance for audit data integrity reviews

An incident on the data plane (query latency spike, ingestion lag) is paged immediately. An incident on the control plane (ontology change went wrong, registry corruption) is paged but the response is slower-paced because the data plane keeps running on cached state — buying time to investigate and recover.

What this implies for security boundaries

- mTLS + SPIFFE workload identity between all services regardless of plane.
- Network policies on OpenShift segregate namespaces per service.
- The two privileged Neo4j databases (privileged identity, executive systems — see §8) get their own network namespaces with stricter ingress allowlists.
- Control plane registries get tighter access controls: only registered service principals can read; only CAB-approved processes can write.
- Audit writes use a separate authenticated path; the audit store is the only component the data plane writes to that has no read-back from the data plane.

— End of document.